HANDS-ON LAB GUIDE · ORACLE RAC 19.32 ON KUBERNETES

Oracle RAC 19c on kubeadm

Rebuild the whole lab by hand, from the Oracle Linux ISO to a 2-node RAC database running in Kubernetes pods. Every step says what to run, where to run it, why it's needed, and what you should see.

`Oracle Linux 9.8 · UEK R7` `VirtualBox 7` `Kubernetes 1.36 · kubeadm · CRI-O` `Flannel + Multus`

`Grid Infrastructure + RAC 19.32`

24 Oct 2026 · Follows Oracle's How to Install and Configure Oracle RAC on Oracle Cloud Native Environment, with every deviation explained.

`PowerShell / Git Bash`

on your PC

`golden VM · controlplane · worker`

root shell on a VM

`racnode1-0 · racnode2-0`

root shell in a RAC pod

How to use this guide

This guide rebuilds the whole lab by hand, from the Oracle Linux ISO to a 2-node Oracle RAC 19.32 database running in Kubernetes pods. Every step has the same four parts:

- **What:** the action, in one line.
- **Why:** what it is for, and what breaks without it. The reasons come from the Oracle doc and from what actually failed during the first build.
- **Commands:** copy and paste as they are.
- **Check:** what you should see before moving on.

Every block runs exactly as written. There's nothing to fill in or change: node names, addresses and disk IDs are either written out for each node or found by the block itself. Where a label says `worker1, then worker2`, paste the same block on each. Work through the guide in order, from 0.0 to the end; each step relies on the ones before it.

The automated scripts in `rac-k8s-poc` do the same work. Use them to rebuild quickly; use this guide to understand it.

Where each command runs

Every command block is labelled with where to paste it. Blocks that run on a VM or in a pod also show how to get there (`connect: ...`); in the HTML guide that command has its own Copy button.

LABEL	WHERE	HOW TO GET THERE
`# PowerShell`	Windows PowerShell on the PC	Start menu, Windows PowerShell
`# PowerShell (Admin)`	PowerShell as Administrator	Right-click, Run as administrator
`# Git Bash`	Git Bash on the PC	Start menu, Git Bash
`# golden VM`	root shell on the golden VM	`ssh -p 2205 root@127.0.0.1`
`# controlplane`, `# worker1`, `# worker2`	root shell on that node	`ssh -p 2210` / `2211` / `2212 root@127.0.0.1`
`# worker1 (still on 10.0.1.5)`	that clone during Phase 4, before it moves to its own address	`ssh -p 2205 root@127.0.0.1`
`# worker1, then worker2`	the same block on each worker in turn	one SSH session per worker
`# racnode1-0`, `# racnode2-0`	root shell inside that RAC pod	from the controlplane: `kubectl -n rac exec -it racnode1-0 -- bash`

Two traps cost time in the first build:

- **PowerShell is not bash.** `\"`, `<<'EOF'` and `$VAR` mean different things. Blocks marked PowerShell are written for PowerShell; long Linux scripts are sent to a node with a here-string piped into `ssh` (`$x | ssh ... "tr -d '\r' | bash -s"`), which avoids PowerShell's quoting entirely.
- **Git Bash needs `./` paths and `winpty`** for programs that ask for a password (`winpty python scan_test.py`). `.\file` turns into `.file` there.

Lab values

These are the values from the lab this guide was written on. Change them if yours differ; everything else follows from them.

ITEM	VALUE
Windows host	64 GB RAM, 8 cores / 8 threads, VirtualBox 7
VM folder	`D:\VMs\rac-poc`
Oracle Linux ISO	`D:\ISO images\OracleLinux-R9-U8-x86_64-dvd-20260618.iso`
SSH key	`%USERPROFILE%\.ssh\id_ed25519`
NAT Network `k8s-public`	10.0.1.0/24, DHCP off
Golden VM	10.0.1.5, SSH port 2205
controlplane / worker1 / worker2	10.0.1.10 / .11 / .12, SSH ports 2210 / 2211 / 2212
RAC private networks	`rac-priv1` 10.0.11.0/24, `rac-priv2` 10.0.12.0/24

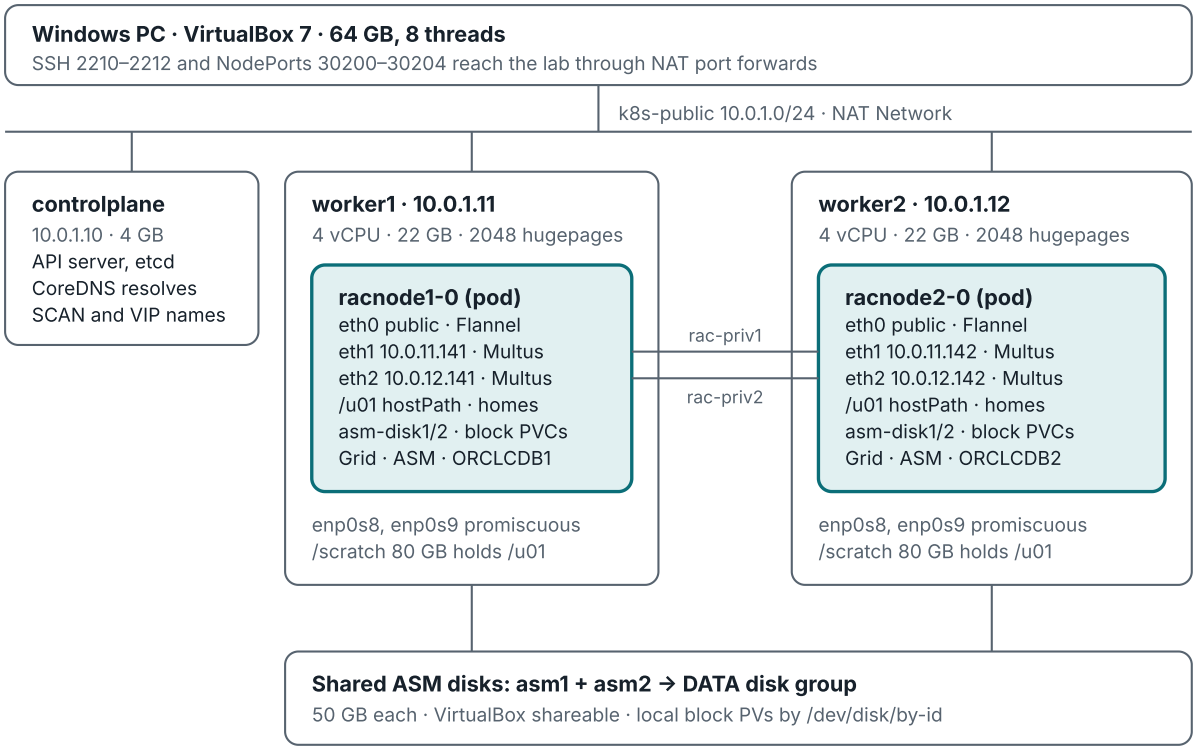
ITEM	VALUE
Pod network (Flannel)	10.244.0.0/16
Oracle software	`oracle-gridhome-19.32.zip`, `oracle-dbhome-19.32.zip` (gold images, fully patched)
PC LAN address (software download)	192.168.1.160

Passwords are never written in this guide or in any file you keep. Where one is needed you type it at a prompt; it lives only inside the pod and is removed after use.

Architecture

Each RAC node is a Kubernetes pod pinned to its own worker VM. Each pod has three networks and two shared disks, and its Oracle software sits on its worker's own disk.

Two RAC pods share the ASM disks and talk over two private networks



Lab architecture: 3 VMs, 2 RAC pods, 3 networks, 2 shared disks.

How to read it:

- **The public network** (`k8s-public`) carries Kubernetes, Flannel's pod traffic and client connections. Each pod's `eth0` is the RAC public network, and its node name, VIP and SCAN all resolve to that address through

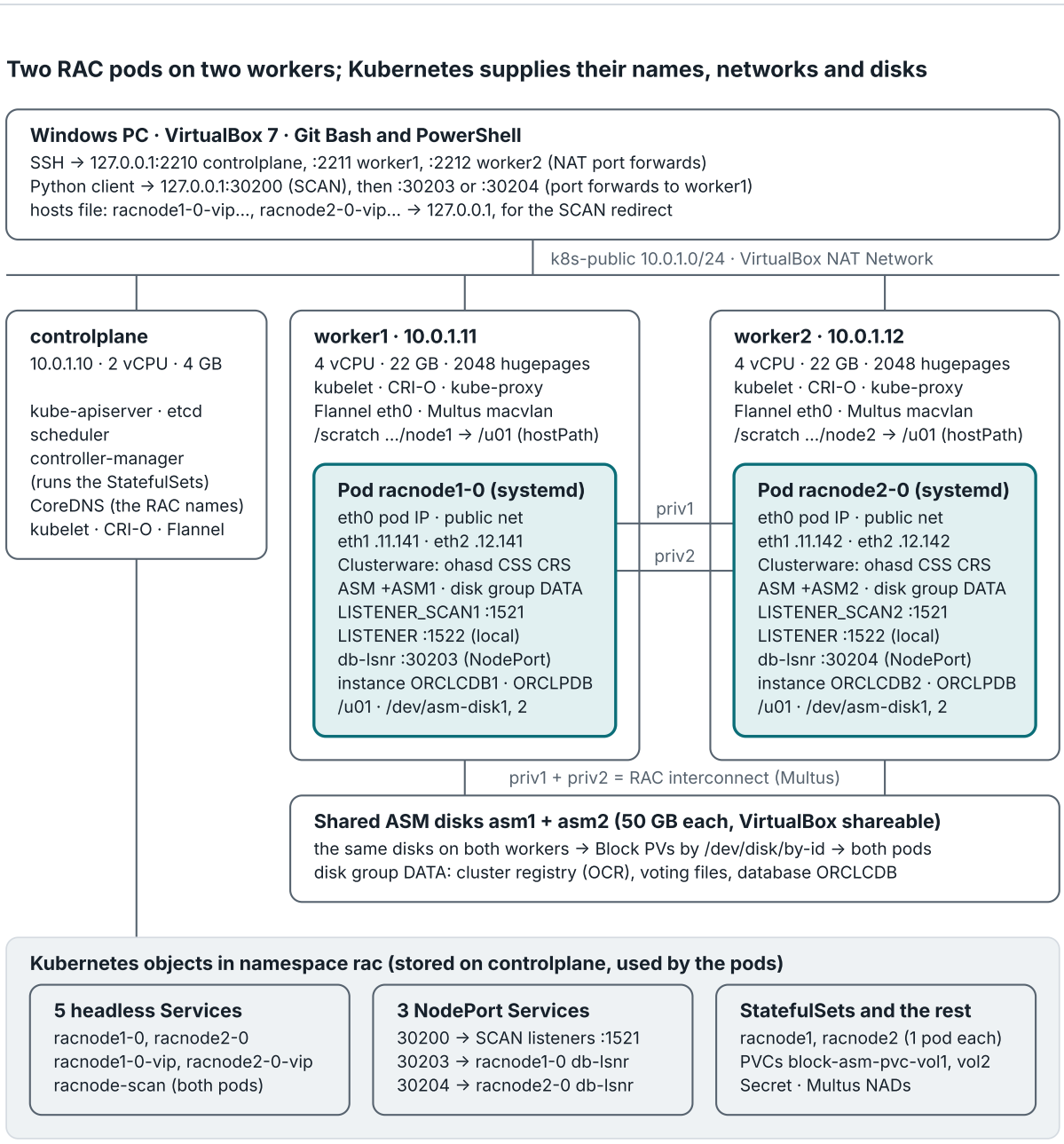
CoreDNS.

- **The two private networks** (`rac-priv1` , `rac-priv2`) are the RAC interconnect. Multus gives each pod `eth1` and `eth2` on them, through macvlan on the workers' promiscuous NICs.
- **The ASM disks** are attached to both worker VMs and handed to both pods as raw block devices. ASM builds the `DATA` disk group on them, which holds the database, the cluster registry and the voting files.
- `/u01` **in each pod** is a directory on its worker's `/scratch` disk. The Oracle homes and the node's local configuration live there and survive the pod being recreated.

LAYER	WHAT	WHERE IT'S BUILT
Windows host	VirtualBox 7, NAT Network, port forwards	Phase 0
VMs	Oracle Linux 9.8, UEK R7, CRI-O, kubeadm	Phases 1–4
Kubernetes	v1.36, Flannel, Multus	Phases 5–6
Storage	Shared disks as block PVs; `/u01` as hostPath	Phases 3, 4, 7
RAC pods	Image, headless Services, StatefulSets	Phases 8–10
Oracle	Grid 19.32, ASM `DATA` , RAC database `ORCLCDB`	Phases 11–12
Operations	SCAN and NodePort access, restart recovery	Phases 13–14

The whole system in one picture

The diagram above shows where things run. This one adds what runs where: the Kubernetes components on each VM, the Oracle processes in each pod, the Kubernetes objects that tie them together, and how you reach the lab from Windows.



The whole lab in one picture. Pod IPs change when a pod is recreated; the private IPs and names do not.

- **Top to bottom** is the path into the lab: Windows reaches everything through VirtualBox port forwards on `127.0.0.1`, and those land on the `k8s-public` network.
- **controlplane** runs Kubernetes itself. It runs no RAC pod; it keeps the objects at the bottom and answers DNS for the RAC names.
- **Each worker** runs one RAC pod. Inside, systemd starts Grid Infrastructure (Clusterware, ASM, the listeners) and the database instance, like on a physical server.
- **The bottom band** lists the Kubernetes objects that stand in for a server room's DNS, network and storage set-up. The Services, StatefulSet and storage companion guides explain each one.

PHASE 0

Windows host and VirtualBox networks

Result of this phase: an SSH key, an SSH configuration for the lab, and the VirtualBox networks every VM will use.

0.0 Start clean (only if this PC had the lab before)

What: removes everything an earlier build of this lab left behind: the four VMs (including the golden VM), their disks and the shared ASM disks, the `k8s-public` network, the SSH host keys remembered for the lab's ports, and the lab's lines in the Windows hosts file. On a PC that never had the lab, both blocks just report that there's nothing to remove.

Why: a rebuild trips over leftovers. VirtualBox refuses to create a VM, disk or network whose name already exists; old ASM disks still carry the previous cluster's ASM headers; and SSH refuses the new VMs' host keys on the same ports ("REMOTE HOST IDENTIFICATION HAS CHANGED").

Only the lab's own names are touched: `ol98-golden`, `controlplane`, `worker1`, `worker2`, the folder `D:\VMs\rac-poc` and the network `k8s-public`. Other VMs are left alone.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-poc"
$existing = (& $VB list vms) -join "`n"
foreach ($v in "worker1","worker2","controlplane","ol98-golden") {
  if ($existing -match ('"' + $v + '"')) { & $VB controlvm $v poweroff 2>$null; Start-Sleep 3; & $VB
unregistervm $v --delete; "removed VM $v" }
}
$left = (& $VB list hdds) | Select-String '^Location:\s+(.*)$' | ForEach-Object {
$_.Matches[0].Groups[1].Value } | Where-Object { $_ -like "$Base\*" }
foreach ($d in $left) { & $VB closemedium disk $d --delete; "removed disk $d" }
if (Test-Path $Base) { Remove-Item $Base -Recurse -Force; "removed folder $Base" }
if ((& $VB natnetwork list) -match 'k8s-public') { & $VB natnetwork stop --netname k8s-public; & $VB
natnetwork remove --netname k8s-public; "removed network k8s-public" }
foreach ($f in "$env:USERPROFILE\.ssh\known_hosts", "$env:USERPROFILE\.ssh\known_hosts_rac_lab") {
  if (Test-Path $f) { foreach ($p in 2205, 2210, 2211, 2212) { ssh-keygen -R "[127.0.0.1]:$p" -f $f
2>$null | Out-Null } }
}
"--- left over (should list none of the lab VMs, and no k8s-public) ---"
& $VB list vms; & $VB natnetwork list
```

Then the hosts file, which needs PowerShell **as Administrator**. The first line shows what will be removed (nothing, on a PC that never had the lab); a backup is kept as `hosts.bak`.

PowerShell (Admin)

```
$h = "$env:windir\System32\drivers\etc\hosts"
Select-String racnode $h
Copy-Item $h "$h.bak"
(Get-Content $h) | Where-Object { $_ -notmatch 'racnode' } | Set-Content $h -Encoding ascii
```

Check: the list printed at the end of the first block has no `ol98-golden`, `controlplane`, `worker1` or `worker2`, and `natnetwork list` doesn't show `k8s-public`.

0.1 Check the host

Why: two workers of 22 GB, a 4 GB control plane and Windows itself need about 56 GB of the 64 GB. Worker vCPUs come from the host's thread count: with 8 threads, workers get 4 vCPUs and each RAC pod 3 CPUs. Giving 6 vCPUs to each worker on an 8-thread host overcommitted the CPU in the first build.

```powershell
Get-CimInstance Win32_Processor | Select-Object NumberOfCores, NumberOfLogicalProcessors
[Math]::Round((Get-CimInstance Win32_ComputerSystem).TotalPhysicalMemory / 1GB)
Get-PSDrive D | Select-Object Used, Free
```

Check: 8 or more logical processors, about 64 GB RAM, 250 GB or more free on D:. If you can, turn off Hyper-V, WSL2 and Memory Integrity so VirtualBox uses its own hypervisor; under Hyper-V it runs noticeably slower.

0.2 SSH key and lab SSH settings

Why: every VM gets this public key at install, so you log in without a password. All VMs are reached through `127.0.0.1` on different ports, and their host keys change when the golden image is sealed and cloned; the settings below stop SSH refusing those changes. They are for this lab only.

```powershell
if (-not (Test-Path "$env:USERPROFILE\.ssh\id_ed25519")) { ssh-keygen -t ed25519 }   # press Enter at every prompt
$cfg = "$env:USERPROFILE\.ssh\config"
if (-not ((Test-Path $cfg) -and (Select-String -Path $cfg -Pattern '^Host 127\.0\.0\.1$' -Quiet))) {
@'

Host 127.0.0.1
    User root
    StrictHostKeyChecking no
    UserKnownHostsFile ~/.ssh/known_hosts_rac_lab
    LogLevel ERROR
    ServerAliveInterval 30
'@ | Add-Content -Encoding ascii $cfg
}
Get-Content "$env:USERPROFILE\.ssh\id_ed25519.pub"
Select-String -Path $cfg -Pattern '^Host 127\.0\.0\.1$' -Context 0,5
```

The key is only created if you don't have one yet, and the settings are only added once, so the block is safe to run again.

Check: one `ssh-ed25519 ...` line (your public key), then the `Host 127.0.0.1` section with its five settings.

0.3 The VirtualBox networks

What: one NAT Network for the public side, SSH port forwards into it, and two internal networks for the RAC interconnect.

Why:

- `k8s-public` **(10.0.1.0/24):** every VM on one network with internet access through NAT. Kubernetes, Flannel's pod traffic and later the RAC public network all run here. DHCP is off because every node gets a fixed address; Kubernetes and Grid both depend on addresses not changing.
- **Port forwards:** the NAT Network can't be reached from Windows directly, so `127.0.0.1:2205/2210/2211/2212` are forwarded to SSH on each VM.
- `rac-priv1` **and** `rac-priv2` **:** two isolated internal networks for the RAC private interconnect (cache fusion and the cluster heartbeat). Two of them give Grid's HAIP redundancy. VirtualBox creates internal networks when a NIC is attached to them, so there is nothing to create here.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
& $VB natnetwork add --netname k8s-public --network "10.0.1.0/24" --enable --dhcp off
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-golden:tcp:[]:2205:[10.0.1.5]:22"
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-cp:tcp:[]:2210:[10.0.1.10]:22"
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-w1:tcp:[]:2211:[10.0.1.11]:22"
& $VB natnetwork modify --netname k8s-public --port-forward-4 "ssh-w2:tcp:[]:2212:[10.0.1.12]:22"
& $VB natnetwork start --netname k8s-public
& $VB natnetwork list
```

Check: `natnetwork list` shows `k8s-public` , `10.0.1.0/24` , DHCP disabled, and the four `ssh-*` rules.

PHASE 1

Golden VM and unattended Oracle Linux 9.8 install

Result of this phase: a VM called `ol98-golden` with a minimal Oracle Linux 9.8 install, reachable as `ssh -p 2205 root@127.0.0.1` .

Why a golden image: every node needs the same kernel, packages, kernel settings and container runtime. Preparing them once and cloning three times is faster and guarantees the nodes match; RAC and Kubernetes both misbehave when nodes differ.

1.1 Create the VM

Why each setting:

- **2 vCPU, 4 GB, 50 GB disk:** enough to install and prepare; each clone is resized later.
- `--paravirtprovider kvm` **:** Oracle Linux's UEK kernel runs best with KVM paravirtualisation (stable clock, lower overhead).
- **NIC1 on** `k8s-public` **; NIC2 and NIC3 on** `rac-priv1` **and** `rac-priv2` **with promiscuous mode** `allow-all` **.** RAC pods will put their own MAC addresses on these NICs through macvlan, and VirtualBox only delivers frames for unknown MACs in allow-all mode. Without it, the interconnect drops every packet.
- **Fixed SATA ports:** port 0 for the OS disk, ports 1 and 2 later for the worker data disks, 3 and 4 for the shared ASM disks, 5 for the DVD.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
$Base = "D:\VMs\rac-poc"; $Iso = "D:\ISO images\OracleLinux-R9-U8-x86_64-dvd-20260618.iso"; $g = "ol98-golden"
New-Item -ItemType Directory -Force -Path $Base | Out-Null
& $VB createvm --name $g --ostype Oracle_64 --basefolder $Base --register
& $VB modifyvm $g --cpus 2 --memory 4096 --vram 16 --graphicscontroller vmsvga --ioapic on --paravirtprovider kvm --rtcuseutc on --boot1 dvd --boot2 disk --boot3 none --boot4 none
& $VB modifyvm $g --nic1 natnetwork --nat-network1 k8s-public
& $VB modifyvm $g --nic2 intnet --intnet2 rac-priv1 --nicpromisc2 allow-all
& $VB modifyvm $g --nic3 intnet --intnet3 rac-priv2 --nicpromisc3 allow-all
& $VB storagectl $g --name SATA --add sata --controller IntelAhci --portcount 6 --hostiocache off
& $VB createmedium disk --filename "$Base\$g\$g-os.vdi" --size 51200 --variant Standard
& $VB storageattach $g --storagectl SATA --port 0 --device 0 --type hdd --medium "$Base\$g\$g-os.vdi"
& $VB storageattach $g --storagectl SATA --port 5 --device 0 --type dvddrive --medium $Iso
```

1.2 Kickstart and unattended install

What: a kickstart file answers every installer question, and `VBoxManage unattended install` boots the ISO with it.

Why these kickstart choices:

- **Static IP 10.0.1.5:** the temporary address of the golden VM. Each clone starts on it and moves to its own address in Phase 4.
- **Your SSH key for root:** passwordless access from the start.
- **SELinux enforcing, firewall off:** the Oracle doc runs SELinux enforcing, with its own policy module for the RAC pods. The firewall is off to keep the lab simple (in production you would open the Kubernetes and Oracle ports instead).
- **One logical volume for `/`, no swap partition:** Kubernetes requires swap off.

The password is typed at a prompt and only exists in a temporary file that is deleted straight after.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Iso = "D:\ISO images\OracleLinux-R9-U8-x86_64-dvd-20260618.iso"; $g = "ol98-golden"
$sec = Read-Host "Root password for the lab VMs (8+ chars, no spaces or double quotes)" -AsSecureString
$pw  = [Runtime.InteropServices.Marshal]::PtrToStringAuto([Runtime.InteropServices.Marshal]::SecureStringToBSTR($sec))
$pub = (Get-Content "$env:USERPROFILE\.ssh\id_ed25519.pub" -Raw).Trim()
$ks = @"
text
cdrom
lang en_US.UTF-8
keyboard us
timezone America/New_York --utc
network --device=enp0s3 --bootproto=static --ip=10.0.1.5 --netmask=255.255.255.0 --gateway=10.0.1.1 --nameserver=8.8.8.8 --noipv6 --onboot=yes --activate
network --hostname=ol98-golden
rootpw --plaintext "$pw" --allow-ssh
sshkey --username=root "$pub"
selinux --enforcing
firewall --disabled
firstboot --disabled
skipx
ignoredisk --only-use=sda
zerombr
clearpart --all --initlabel --drives=sda
reqpart --add-boot
part pv.01 --size=1 --grow --ondisk=sda
volgroup ol pv.01
logvol / --vgname=ol --name=root --fstype=xfs --size=1 --grow
reboot --eject

%packages
@^minimal-environment
chrony
tar
rsync
%end
"@
$ksFile = "$env:TEMP\ol98-golden-ks.cfg"
[IO.File]::WriteAllText($ksFile, $ks.Replace("`r`n", "`n"), (New-Object Text.UTF8Encoding $false))
& $VB unattended install $g "--iso=$Iso" "--script-template=$ksFile" --hostname=ol98-golden.localdomain --user=rac "--password=$pw" --time-zone=America/New_York --locale=en_US --country=US --start-vm=headless
Remove-Item $ksFile; $pw = $null
```

The install takes 10 to 20 minutes. To watch it, open VirtualBox Manager, select `ol98-golden` and click **Show**.

Check (when SSH answers; until then you get "connection refused" or a timeout):

PowerShell

```
ssh -p 2205 root@127.0.0.1 "cat /etc/oracle-release; uname -r; ip -4 -br addr show enp0s3; swapon --show
| wc -l; getenforce"
```

You should see `Oracle Linux Server release 9.8`, `enp0s3 ... 10.0.1.5/24`, `0` swap devices and `Enforcing`. The kernel is still the installer's default here; Phase 2 pins UEK R7.

PHASE 2

Golden VM OS prep for Kubernetes and RAC

Result of this phase: the golden VM has everything all three nodes share: the UEK R7 kernel, the kernel settings Oracle and Kubernetes need, the SELinux policy for RAC pods, and CRI-O with kubeadm, kubelet and kubectl. Run every block in a root shell on the golden VM:

PowerShell

```
ssh -p 2205 root@127.0.0.1
```

2.1 Update the OS and pin the UEK R7 kernel

Why: the Oracle doc is tested on UEK R7 (5.15.0-208 or later). Oracle Linux 9.8 may default to UEK 8, so the newest UEK R7 kernel is installed and made the default. Running a kernel the doc wasn't tested on is a variable a PoC doesn't need.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
dnf -y install dnf-plugins-core
dnf -y update
dnf config-manager --set-enabled ol9_UEKR7
dnf config-manager --set-disabled ol9_UEKR8 2>/dev/null || true
UEK7=$(dnf -q --disablerepo='*' --enablerepo=ol9_UEKR7 repoquery --latest-limit 1 --qf '%{version}-%
{release}' kernel-uek | tail -1)
echo "UEK R7: $UEK7"
dnf -y install "kernel-uek-${UEK7}"
grubby --set-default "/boot/vmlinuz-${UEK7}.x86_64"
```

2.2 Kernel arguments

Why:

- `transparent_hugepage=madvise`: Oracle recommends against THP `always`, which causes memory-access delays and, in RAC, node evictions. The doc asks for `madvise` on UEK R7.
- `clocksource=tsc`: a stable clock in a VM. Grid's cluster time and its heartbeat timing are sensitive to clock jumps.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
grubby --update-kernel=ALL --args="transparent_hugepage=madvise clocksource=tsc"
```

2.3 Swap off

Why: the kubelet refuses to run with swap on by default. Memory limits on pods only mean something when nothing is swapped out.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
swapoff -a
sed -ri '/\sswap\s/ s/^([^#])/#\1/' /etc/fstab
```

2.4 Kernel modules and sysctls

Why:

- `overlay` **and** `br_netfilter` **, plus the three** `net.*` **settings:** container image layers, and traffic between pods passing through iptables. Kubernetes networking needs them.
- **The Oracle settings:** the doc's values for the worker hosts (`fs.file-max` , socket buffers, `fs.aio-max-nr`). These are host-wide settings that pods can't set themselves. The settings pods *can* set for themselves (`kernel.shm*` , `kernel.sem`) come later, in the pod definitions.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
cat >/etc/modules-load.d/k8s.conf <<'EOF'
overlay
br_netfilter
EOF
modprobe overlay; modprobe br_netfilter
cat >/etc/sysctl.d/90-k8s.conf <<'EOF'
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
cat >/etc/sysctl.d/91-oracle-rac.conf <<'EOF'
fs.file-max = 6815744
net.core.rmem_default = 262144
net.core.rmem_max = 4194304
net.core.wmem_default = 262144
net.core.wmem_max = 1048576
fs.aio-max-nr = 1048576
EOF
sysctl --system >/dev/null
```

2.5 Time sync and firewall

Why: all nodes must agree on the time. Grid checks it, and Kubernetes certificates and leases depend on it. The firewall is off for the lab only.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
dnf -y install chrony
systemctl enable --now chronyd
systemctl disable --now firewalld 2>/dev/null || true
```

2.6 Tools and the RAC SELinux policy

Why:

- **Tools:** `podman` builds the RAC image; `unzip` unpacks the Oracle software. It was missing in the first build, and the Grid unzip step failed until it was installed.
- **The `rac-ocne` policy module:** RAC pods run systemd as PID 1 in the SELinux domain `container_init_t`. Oracle binaries on `/u01` need to map and execute files, use text relocations and read the kernel log, which the default container policy forbids. This module is the doc's "Set up SELinux Module on Worker Nodes".
- `container_manage_cgroup`**:** systemd inside a pod must manage its own cgroups. Without this boolean, the RAC container exited about one second after starting, with no log output. OCNE sets it through its own SELinux package; on kubeadm you set it yourself.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
dnf -y install container-selinux selinux-policy-devel policycoreutils-python-utils podman tar rsync unzip
jq iproute-tc bash-completion lsof
mkdir -p /var/opt && cd /var/opt
cat >rac-ocne.te <<'EOF'
module rac-ocne 1.0;

require {
        type container_init_t;
        type container_runtime_t;
        type container_file_t;
        type kernel_t;
        type bin_t;
        type lib_t;
        type textrel_shlib_t;
        class file { getattr map execute execmod setattr };
        class system syslog_read;
}

allow container_init_t container_runtime_t:file getattr;
allow container_init_t bin_t:file { map execute };
allow container_init_t container_file_t:file execmod;
allow container_init_t lib_t:file execmod;
allow container_init_t textrel_shlib_t:file setattr;
allow container_init_t kernel_t:system syslog_read;
EOF
make -f /usr/share/selinux/devel/Makefile rac-ocne.pp && semodule -i rac-ocne.pp
setsebool -P container_manage_cgroup on
cd ~
```

Check: `semodule -l | grep rac-ocne` prints `rac-ocne`, and `getsebool container_manage_cgroup` prints `on`.

2.7 Kubernetes and CRI-O packages

What: the Kubernetes v1.36 and CRI-O v1.36 repositories, then `cri-o`, `kubelet`, `kubeadm`, `kubectl` and `cri-tools`.

Why CRI-O: the doc builds the RAC image with podman and runs it as `localhost/oracle/database-rac`. CRI-O shares podman's image store, so a podman-built image is immediately visible to Kubernetes with no registry. With containerd you would have to import it separately. `cri-tools` gives `crictl`, which was also missing at first.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
cat >/etc/yum.repos.d/kubernetes.repo <<'EOF'
[kubernetes]
name=Kubernetes
baseurl=https://pkgs.k8s.io/core:/stable:/v1.36/rpm/
enabled=1
gpgcheck=1
gpgkey=https://pkgs.k8s.io/core:/stable:/v1.36/rpm/repodata/repomd.xml.key
exclude=kubelet kubeadm kubectl cri-tools kubernetes-cni
EOF
cat >/etc/yum.repos.d/cri-o.repo <<'EOF'
[cri-o]
name=CRI-O
baseurl=https://download.opensuse.org/repositories/isv:/cri-o:/stable:/v1.36/rpm/
enabled=1
gpgcheck=1
gpgkey=https://download.opensuse.org/repositories/isv:/cri-o:/stable:/v1.36/rpm/repodata/repomd.xml.key
EOF
dnf -y install cri-o kubelet kubeadm kubectl cri-tools --disableexcludes=kubernetes
echo "runtime-endpoint: unix:///var/run/crio/crio.sock" >/etc/crictl.yaml
```

The `exclude=` line stops a routine `dnf update` from upgrading Kubernetes by accident. Kubernetes is upgraded deliberately, one minor version at a time.

2.8 Configure CRI-O and pull the images

Why:

- `selinux = true` : containers get SELinux labels, which the `rac-ocne` module is written for.
- **The pause image:** set to the exact image kubeadm expects. Otherwise kubeadm warns about a mismatch, and two pause images get pulled.
- **No CRI-O sample bridge network:** Flannel and Multus provide the pod networks.
- **Images pulled now:** each clone then starts with the control-plane images already in place.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
mkdir -p /etc/crio/crio.conf.d
printf '[crio.runtime]\nselinux = true\n' >/etc/crio/crio.conf.d/90-rac-poc.conf
PAUSE_IMG=$(kubeadm config images list 2>/dev/null | grep '/pause:' | head -1)
printf '[crio.image]\npause_image = "%s"\n' "$PAUSE_IMG" >/etc/crio/crio.conf.d/91-pause.conf
rm -f /etc/cni/net.d/*crio*bridge* 2>/dev/null || true
systemctl enable --now crio
systemctl enable kubelet
kubeadm config images pull --cri-socket unix:///var/run/crio/crio.sock
```

The kubelet is enabled but won't start properly until `kubeadm init` or `join` gives it a configuration. That's expected.

2.9 Host names and shell completion

Why: every node can resolve the others by name before DNS exists. Completion saves typing.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
cat >>/etc/hosts <<'EOF'
10.0.1.10  controlplane
10.0.1.11  worker1
10.0.1.12  worker2
EOF
kubectl completion bash >/etc/bash_completion.d/kubectl
crictl completion bash >/etc/bash_completion.d/crictl
```

2.10 Reboot and check

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
systemctl reboot
```

Reconnect after a minute, then:

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
uname -r
cat /sys/kernel/mm/transparent_hugepage/enabled
cat /sys/devices/system/clocksource/clocksource0/current_clocksource
swapon --show | wc -l
getenforce
systemctl is-active crio
crictl images | wc -l
kubeadm version -o short
```

Check:

- the kernel is `5.15.0-...el9uek`;
- THP shows `[madvise]`, and the clock source is `tsc`;
- swap is `0`, and SELinux is `Enforcing`;
- CRI-O is `active`;
- there are at least 7 images, and kubeadm is `v1.36.x`.

PHASE 3

Seal the golden image and clone the three nodes

Result of this phase: three VMs (`controlplane` , `worker1` , `worker2`) cloned from the golden image, the workers with two data disks each, and two 50 GB disks that **both** workers share for ASM.

3.1 Snapshot, then seal

Why:

- **The snapshot** (`pre-seal`) lets you go back and fix the golden image without reinstalling.
- **Sealing** removes everything that must be unique per machine: the machine ID, the SSH host keys, the logs. Kubernetes refuses to join two nodes with the same machine ID, and identical SSH host keys would make every node look the same to SSH.
- **VirtualBox gives each full clone a new hardware UUID and new MAC addresses**, which kubeadm also checks.

golden VM

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
systemctl poweroff
```

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $g = "ol98-golden"
while ("$(& $VB showvminfo $g --machinereadable | Select-String '^VMState=')" -notmatch 'poweroff') {
Start-Sleep 3 }
& $VB storageattach $g --storagectl SATA --port 5 --device 0 --type dvddrive --medium emptydrive
& $VB snapshot $g take pre-seal --description "Prepared golden image before sealing"
& $VB startvm $g --type headless
```

If `startvm` says the VM is locked, wait 10 seconds and run it again; VirtualBox holds a lock briefly after a snapshot.

golden VM

```
(the last command powers it off; don't start it again before cloning)
```

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
dnf clean all
rm -f /etc/ssh/ssh_host_*
truncate -s 0 /etc/machine-id
rm -f /var/lib/dbus/machine-id
rm -rf /tmp/* /var/tmp/*
journalctl --rotate; journalctl --vacuum-time=1s
rm -f /root/.bash_history; history -c
poweroff
```

3.2 Clone and size the nodes

Why these sizes:

- **controlplane, 2 vCPU and 4 GB:** runs only the Kubernetes control plane. It has no private NICs, because no RAC pod ever runs there.
- **Workers, 4 vCPU and 22 GB:** each runs one RAC pod with a 12 GiB memory limit, plus 4 GB of hugepages, plus the node's own processes.
- **An 80 GB disk per worker (`/scratch`):** holds `/u01` for the RAC pod (the Oracle homes and diagnostic files) and the staged software.
- **A 100 GB disk per worker (`/var/lib/containers`):** container images and their writable layers, kept off the OS disk.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-poc"; $g = "ol98-golden"
while ("$(& $VB showvminfo $g --machinereadable | Select-String '^VMState=')" -notmatch 'poweroff') {
Start-Sleep 3 }
foreach ($n in "controlplane","worker1","worker2") { & $VB clonevm $g --name $n --basefolder $Base --mode
machine --register }
& $VB modifyvm controlplane --cpus 2 --memory 4096 --nic2 none --nic3 none
foreach ($w in "worker1","worker2") {
  & $VB modifyvm $w --cpus 4 --memory 22528
  & $VB createmedium disk --filename "$Base\$w\$w-scratch.vdi" --size 81920 --variant Standard
  & $VB createmedium disk --filename "$Base\$w\$w-containers.vdi" --size 102400 --variant Standard
  & $VB storageattach $w --storagectl SATA --port 1 --device 0 --type hdd --medium "$Base\$w\$w-
scratch.vdi"
  & $VB storageattach $w --storagectl SATA --port 2 --device 0 --type hdd --medium "$Base\$w\$w-
containers.vdi"
}
```

3.3 The shared ASM disks

What: two 50 GB disks attached to **both** workers at once.

Why:

- **RAC is a shared-disk database.** Both instances read and write the same data files, so both nodes need the same block devices. On real hardware that's a SAN; here it's VirtualBox shareable disks.
- **Fixed size:** VirtualBox only allows the shareable type on fixed-size disks. Creating them takes a few minutes each.
- **Mode `shareable`:** without it, VirtualBox refuses to attach the second VM, or worse, caches writes per VM.
- **One consequence:** VirtualBox can't snapshot a VM with shareable disks attached, so don't snapshot the workers.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-poc"
New-Item -ItemType Directory -Force -Path "$Base\shared" | Out-Null
$port = 3
foreach ($d in "asm1","asm2") {
  & $VB createmedium disk --filename "$Base\shared\$d.vdi" --size 51200 --variant Fixed
  & $VB modifymedium disk "$Base\shared\$d.vdi" --type shareable
  foreach ($w in "worker1","worker2") { & $VB storageattach $w --storagectl SATA --port $port --device 0
--type hdd --medium "$Base\shared\$d.vdi" --mtype shareable }
  $port++
}
foreach ($d in "asm1","asm2") { & $VB showmediuminfo disk "$Base\shared\$d.vdi" }
```

Check: for **each** disk, `showmediuminfo` shows `Type: shareable`, and under "In use by VMs" both `worker1` and `worker2`. Note the first 8 characters of its UUID: inside the workers the disk appears as `/dev/disk/by-id/ata-VBOX_HARDDISK_VB<those 8>-...`.

PHASE 4

Personalize each node

Result of this phase: each node has its own name and IP address. The workers also have their private NICs ready for Multus, `/scratch` and `/var/lib/containers` on their own disks, the doc's directories, and 4 GB of hugepages.

Do one node at a time, in the order controlplane, worker1, worker2. Every clone boots with the golden VM's address 10.0.1.5, and two clones running at once would fight over it. Each node has its own section below with every command written out for it: copy the blocks in order, without changing anything. Start the next node only after the previous one passes its last check.

4.1 What happens to each node, and why

STEP	CONTROLPLANE	WORKER1, WORKER2
a. Start and connect on 2205	yes	yes
b. Name, identity, kubelet IP	yes	yes
c. Private NICs for Multus	no: it has no NIC2/NIC3	yes
d, e. Data disks `/var/lib/containers` and `/scratch`	no: it has only the OS disk	yes
f. The doc's directories	no	yes (different on each worker)
g. HugePages	no: 2048 pages would lock all of its 4 GB	yes

STEP	CONTROLPLANE	WORKER1, WORKER2
Move to its own address, then check it	yes	yes

Why each step:

- **Name, identity and kubelet IP:** the host name becomes the Kubernetes node name. New SSH host keys replace the ones the seal removed. `--node-ip` matters on the workers, which have three NICs: without it the kubelet may advertise the wrong address, and pod traffic between nodes breaks.
- **Private NICs with no IP:** `enp0s8` and `enp0s9` are only "parents" for Multus macvlan interfaces. Each RAC pod gets its own interface, IP (10.0.11.x and 10.0.12.x) and MAC address on top of them.
- **Promiscuous mode (`accept-all-mac-addresses yes`):** this was the cause of the hardest network failure in the first build, 100% packet loss between pods. VirtualBox's "Allow All" setting only takes effect when the guest NIC itself is promiscuous. Otherwise the virtual NIC silently drops frames addressed to the pods' macvlan MACs.
- **Data disks chosen by size, not by name:** Linux disk names (`sdb`, `sdc` ...) do **not** follow VirtualBox port order. On worker2 in the first build, `/dev/sdc` was a shared ASM disk, and formatting by name would have wiped the disk both nodes share. The ASM disks are 50 GB, so the sizes tell the disks apart. They're **mounted by UUID** for the same reason, and `rsync -aAXH` copies the images pulled in Phase 2 together with their SELinux labels.
- **The doc's directories with `container_file_t`:** they're mounted into pods as hostPath volumes. With SELinux enforcing, a container can only use host files with the container file label. `semanage fcontext` makes the label permanent.
- **HugePages:** the database SGA goes into 2 MB hugepages, which are locked, never swapped and cheaper for the CPU to map. Kubernetes only gives hugepages to pods on nodes that reserved them: the RAC pods ask for `hugepages-2Mi: 4Gi`, so each worker reserves 2048 pages. The doc reserves 16384 (32 GB); this lab scales it to the 22 GB workers.
- **Moving the address last, in the background:** changing the public IP cuts the SSH session you're typing in. A first attempt died halfway through for exactly that reason. So the change runs detached from your session and ends with a reboot.
- `ssh-keygen -R` **before connecting:** every clone has its own host key but answers on the same port, 2205. Without the 0.2 SSH settings, SSH refuses the new key with "REMOTE HOST IDENTIFICATION HAS CHANGED"; removing the old entry avoids that.

4.2 controlplane

a. Start it and connect (port 2205 reaches whichever clone has 10.0.1.5):

PowerShell

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" startvm controlplane --type headless
ssh-keygen -R "[127.0.0.1]:2205" 2>$null
ssh -p 2205 root@127.0.0.1
```

If SSH says "connection refused" or times out, the VM is still booting: wait 30 seconds and run the `ssh` line again. Answer `yes` to the host key question. **Check:** `hostname` says `ol98-golden`.

b. Name, identity and kubelet IP:

controlplane (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
hostnamectl set-hostname controlplane
[ -s /etc/machine-id ] || systemd-machine-id-setup
ssh-keygen -A
echo 'KUBELET_EXTRA_ARGS="--node-ip=10.0.1.10"' >/etc/sysconfig/kubelet
echo "machine-id $(cat /etc/machine-id)  product_uuid $(cat /sys/class/dmi/id/product_uuid)"
hostname; cat /etc/sysconfig/kubelet
```

Check: `controlplane`, `--node-ip=10.0.1.10`, and a machine ID and product UUID different from the other nodes'.

c. Move to 10.0.1.10 and reboot (always last):

controlplane (still on 10.0.1.5; the session freezes or closes at the end)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ "$(hostname)" = "controlplane" ]; then
cat >/root/set-ip.sh <<'EOF'
for u in $(nmcli -g UUID con show); do
  [ "$(nmcli -g connection.interface-name con show $u)" = "enp0s3" ] && nmcli con delete $u
done
nmcli con add type ethernet con-name pub ifname enp0s3 ipv4.method manual ipv4.addresses 10.0.1.10/24 \
  ipv4.gateway 10.0.1.1 ipv4.dns 8.8.8.8 ipv6.method disabled connection.autoconnect yes
sleep 2; reboot
EOF
setsid nohup bash /root/set-ip.sh >/root/set-ip.log 2>&1 </dev/null &
else echo "STOP: this VM is $(hostname), not controlplane; run step b first"; fi
```

Your SSH session freezes or closes. That's expected: the VM is moving to 10.0.1.10 and rebooting.

d. Check it on its own address (wait about a minute after the move):

PowerShell

```
ssh -p 2210 root@127.0.0.1 "hostname; ip -4 -br addr show enp0s3; ip -br link show enp0s8; df -h /scratch
/var/lib/containers; grep HugePages_Total /proc/meminfo; systemctl is-active crio"
```

Check:

- `controlplane`, and `10.0.1.10/24`;
- `Device "enp0s8" does not exist` and `df: /scratch: No such file or directory`: correct, it has no private NICs and no data disks;
- `/var/lib/containers` is on the root filesystem (`/dev/mapper/ol-root`);
- `HugePages_Total: 0`, and CRI-O is `active`.

4.3 worker1

a. Start it and connect (port 2205 reaches whichever clone has 10.0.1.5):

PowerShell

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" startvm worker1 --type headless
ssh-keygen -R "[127.0.0.1]:2205" 2>$null
ssh -p 2205 root@127.0.0.1
```

If SSH says "connection refused" or times out, the VM is still booting: wait 30 seconds and run the `ssh` line again. Answer `yes` to the host key question. **Check:** `hostname` says `ol98-golden`.

b. Name, identity and kubelet IP:

worker1 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
hostnamectl set-hostname worker1
[ -s /etc/machine-id ] || systemd-machine-id-setup
ssh-keygen -A
echo 'KUBELET_EXTRA_ARGS="--node-ip=10.0.1.11"' >/etc/sysconfig/kubelet
echo "machine-id $(cat /etc/machine-id)  product_uuid $(cat /sys/class/dmi/id/product_uuid)"
hostname; cat /etc/sysconfig/kubelet
```

Check: `worker1`, `--node-ip=10.0.1.11`, and a machine ID and product UUID different from the other nodes'.

c. Private NICs, up with no IP and promiscuous:

worker1 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
for ifn in enp0s8 enp0s9; do
  for u in $(nmcli -g UUID con show); do
    [ "$(nmcli -g connection.interface-name con show $u)" = "$ifn" ] && nmcli con delete $u
  done
  nmcli con add type ethernet con-name rac-$ifn ifname $ifn ipv4.method disabled ipv6.method disabled \
    802-3-ethernet.mtu 1500 802-3-ethernet.accept-all-mac-addresses yes connection.autoconnect yes
done
ip -br link show enp0s8; ip -br link show enp0s9
```

Check: both NICs show `UP` and have no IPv4 address. `PROMISC` shows after the reboot in step h.

d. Find the two data disks by size:

worker1 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
lsblk -d -o NAME,SIZE,TYPE
pick() { lsblk -dbno NAME,SIZE,TYPE | awk -v lo=$(( ($1-2)*1024**3 )) -v hi=$(( ($1+2)*1024**3 ))
'$3=="disk" && $2>=lo && $2<=hi {print "/dev/"$1}'; }
CONT=$(pick 100); SCRATCH=$(pick 80); echo "containers=$CONT scratch=$SCRATCH"
[ $(echo "$CONT" | wc -w) -eq 1 ] && [ $(echo "$SCRATCH" | wc -w) -eq 1 ] && [ -z "$(blkid -s TYPE -o
value $CONT $SCRATCH)" ] && echo "OK: two blank disks found" || echo "STOP: check lsblk before
formatting"
```

Check: it prints `OK: two blank disks found`. If it prints STOP, don't go on; paste the `lsblk` output.

e. Format and mount them (same session as d; it formats nothing unless d found the two blank disks):

worker1 (still on 10.0.1.5, same session as step d)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ -b "$CONT" ] && [ -b "$SCRATCH" ] && [ "$CONT" != "$SCRATCH" ] && [ -z "$(blkid -s TYPE -o value
$CONT $SCRATCH)" ]; then
systemctl stop kubelet crio
mkfs.xfs -q -f -L containers $CONT
mkdir -p /mnt/newcont && mount $CONT /mnt/newcont
rsync -aAXH /var/lib/containers/ /mnt/newcont/
umount /mnt/newcont && rmdir /mnt/newcont
echo "UUID=$(blkid -o value -s UUID $CONT)  /var/lib/containers  xfs  defaults  0 0" >>/etc/fstab
mkfs.xfs -q -f -L scratch $SCRATCH
mkdir -p /scratch
echo "UUID=$(blkid -o value -s UUID $SCRATCH)  /scratch  xfs  defaults  0 0" >>/etc/fstab
systemctl daemon-reload
mount /var/lib/containers && mount /scratch
restorecon -R /var/lib/containers
df -h /scratch /var/lib/containers
else echo "STOP: nothing formatted. Run step d in this same session; it must print OK"; fi
```

Check: `/scratch` about 80 G and `/var/lib/containers` about 100 G.

f. The doc's directories, with SELinux labels: `/scratch/rac/cluster01/node1` (racnode1-0's `/u01`), `/scratch/software/stage` (the Oracle zips) and `/scratch/image` (the image build).

worker1 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
for d in /scratch/rac/cluster01/node1 /scratch/software/stage /scratch/image; do
  mkdir -p $d; semanage fcontext -a -t container_file_t "$d(/.*)?"; restorecon -RF $d
done
ls -Zd /scratch/rac/cluster01/node1 /scratch/software/stage /scratch/image
```

Check: each directory shows `container_file_t`.

g. HugePages:

worker1 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
echo "vm.nr_hugepages = 2048" >/etc/sysctl.d/92-hugepages.conf
sysctl -p /etc/sysctl.d/92-hugepages.conf
grep -E 'HugePages_Total|Hugepagesize' /proc/meminfo
```

Check: `HugePages_Total: 2048` and `Hugepagesize: 2048 kB`.

h. Move to 10.0.1.11 and reboot (always last):

worker1 (still on 10.0.1.5; the session freezes or closes at the end)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ "$(hostname)" = "worker1" ]; then
cat >/root/set-ip.sh <<'EOF'
for u in $(nmcli -g UUID con show); do
  [ "$(nmcli -g connection.interface-name con show $u)" = "enp0s3" ] && nmcli con delete $u
done
nmcli con add type ethernet con-name pub ifname enp0s3 ipv4.method manual ipv4.addresses 10.0.1.11/24 \
  ipv4.gateway 10.0.1.1 ipv4.dns 8.8.8.8 ipv6.method disabled connection.autoconnect yes
sleep 2; reboot
EOF
setsid nohup bash /root/set-ip.sh >/root/set-ip.log 2>&1 </dev/null &
else echo "STOP: this VM is $(hostname), not worker1; run step b first"; fi
```

Your SSH session freezes or closes. That's expected: the VM is moving to 10.0.1.11 and rebooting.

i. Check it on its own address (wait about a minute after the move):

PowerShell

```
ssh -p 2211 root@127.0.0.1 "hostname; ip -4 -br addr show enp0s3; ip -br link show enp0s8; df -h /scratch
/var/lib/containers; grep HugePages_Total /proc/meminfo; systemctl is-active crio"
```

Check:

- `worker1`, and `10.0.1.11/24`;
- `enp0s8` shows `UP` and `PROMISC`, with no address;
- `/scratch` is 80 G, and `/var/lib/containers` is 100 G;
- `HugePages_Total: 2048`;
- CRI-O is `active`.

4.4 worker2

a. Start it and connect (port 2205 reaches whichever clone has 10.0.1.5):

PowerShell

```
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" startvm worker2 --type headless
ssh-keygen -R "[127.0.0.1]:2205" 2>$null
ssh -p 2205 root@127.0.0.1
```

If SSH says "connection refused" or times out, the VM is still booting: wait 30 seconds and run the `ssh` line again. Answer `yes` to the host key question. **Check:** `hostname` says `ol98-golden`.

b. Name, identity and kubelet IP:

worker2 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
hostnamectl set-hostname worker2
[ -s /etc/machine-id ] || systemd-machine-id-setup
ssh-keygen -A
echo 'KUBELET_EXTRA_ARGS="--node-ip=10.0.1.12"' >/etc/sysconfig/kubelet
echo "machine-id $(cat /etc/machine-id)  product_uuid $(cat /sys/class/dmi/id/product_uuid)"
hostname; cat /etc/sysconfig/kubelet
```

Check: `worker2`, `--node-ip=10.0.1.12`, and a machine ID and product UUID different from the other nodes'.

c. Private NICs, up with no IP and promiscuous:

worker2 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
for ifn in enp0s8 enp0s9; do
  for u in $(nmcli -g UUID con show); do
    [ "$(nmcli -g connection.interface-name con show $u)" = "$ifn" ] && nmcli con delete $u
  done
  nmcli con add type ethernet con-name rac-$ifn ifname $ifn ipv4.method disabled ipv6.method disabled \
    802-3-ethernet.mtu 1500 802-3-ethernet.accept-all-mac-addresses yes connection.autoconnect yes
done
ip -br link show enp0s8; ip -br link show enp0s9
```

Check: both NICs show `UP` and have no IPv4 address. `PROMISC` shows after the reboot in step h.

d. Find the two data disks by size:

worker2 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
lsblk -d -o NAME,SIZE,TYPE
pick() { lsblk -dbno NAME,SIZE,TYPE | awk -v lo=$(( ($1-2)*1024**3 )) -v hi=$(( ($1+2)*1024**3 ))
'$3=="disk" && $2>=lo && $2<=hi {print "/dev/"$1}'; }
CONT=$(pick 100); SCRATCH=$(pick 80); echo "containers=$CONT scratch=$SCRATCH"
[ $(echo "$CONT" | wc -w) -eq 1 ] && [ $(echo "$SCRATCH" | wc -w) -eq 1 ] && [ -z "$(blkid -s TYPE -o
value $CONT $SCRATCH)" ] && echo "OK: two blank disks found" || echo "STOP: check lsblk before
formatting"
```

Check: it prints `OK: two blank disks found`. If it prints STOP, don't go on; paste the `lsblk` output.

e. Format and mount them (same session as d; it formats nothing unless d found the two blank disks):

worker2 (still on 10.0.1.5, same session as step d)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ -b "$CONT" ] && [ -b "$SCRATCH" ] && [ "$CONT" != "$SCRATCH" ] && [ -z "$(blkid -s TYPE -o value
$CONT $SCRATCH)" ]; then
systemctl stop kubelet crio
mkfs.xfs -q -f -L containers $CONT
mkdir -p /mnt/newcont && mount $CONT /mnt/newcont
rsync -aAXH /var/lib/containers/ /mnt/newcont/
umount /mnt/newcont && rmdir /mnt/newcont
echo "UUID=$(blkid -o value -s UUID $CONT)  /var/lib/containers  xfs  defaults  0 0" >>/etc/fstab
mkfs.xfs -q -f -L scratch $SCRATCH
mkdir -p /scratch
echo "UUID=$(blkid -o value -s UUID $SCRATCH)  /scratch  xfs  defaults  0 0" >>/etc/fstab
systemctl daemon-reload
mount /var/lib/containers && mount /scratch
restorecon -R /var/lib/containers
df -h /scratch /var/lib/containers
else echo "STOP: nothing formatted. Run step d in this same session; it must print OK"; fi
```

Check: `/scratch` about 80 G and `/var/lib/containers` about 100 G.

f. The doc's directories, with SELinux labels: `/scratch/rac/cluster01/node2` (racnode2-0's `/u01`).

worker2 (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
d=/scratch/rac/cluster01/node2
mkdir -p $d; semanage fcontext -a -t container_file_t "$d(/.*)?"; restorecon -RF $d
ls -Zd $d
```

Check: each directory shows `container_file_t`.

g. HugePages:

`worker2` (still on 10.0.1.5)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
echo "vm.nr_hugepages = 2048" >/etc/sysctl.d/92-hugepages.conf
sysctl -p /etc/sysctl.d/92-hugepages.conf
grep -E 'HugePages_Total|Hugepagesize' /proc/meminfo
```

Check: `HugePages_Total: 2048` and `Hugepagesize: 2048 kB`.

h. Move to 10.0.1.12 and reboot (always last):

`worker2` (still on 10.0.1.5; the session freezes or closes at the end)

CONNECT FIRST `ssh -p 2205 root@127.0.0.1`

```
if [ "$(hostname)" = "worker2" ]; then
cat >/root/set-ip.sh <<'EOF'
for u in $(nmcli -g UUID con show); do
  [ "$(nmcli -g connection.interface-name con show $u)" = "enp0s3" ] && nmcli con delete $u
done
nmcli con add type ethernet con-name pub ifname enp0s3 ipv4.method manual ipv4.addresses 10.0.1.12/24 \
  ipv4.gateway 10.0.1.1 ipv4.dns 8.8.8.8 ipv6.method disabled connection.autoconnect yes
sleep 2; reboot
EOF
setsid nohup bash /root/set-ip.sh >/root/set-ip.log 2>&1 </dev/null &
else echo "STOP: this VM is $(hostname), not worker2; run step b first"; fi
```

Your SSH session freezes or closes. That's expected: the VM is moving to 10.0.1.12 and rebooting.

i. Check it on its own address (wait about a minute after the move):

`PowerShell`

```
ssh -p 2212 root@127.0.0.1 "hostname; ip -4 -br addr show enp0s3; ip -br link show enp0s8; df -h /scratch
/var/lib/containers; grep HugePages_Total /proc/meminfo; systemctl is-active crio"
```

Check:

- `worker2`, and `10.0.1.12/24`;
- `enp0s8` shows `UP` and `PROMISC`, with no address;
- `/scratch` is 80 G, and `/var/lib/containers` is 100 G;
- `HugePages_Total: 2048`;
- CRI-O is `active`.

When worker2 passes its check, all three nodes are on their own addresses. Go on to Phase 5.

PHASE 5

Kubernetes with kubeadm and Flannel

Result of this phase: a 3-node Kubernetes v1.36 cluster: the controlplane plus worker1 and worker2, all `Ready`, with Flannel as the pod network.

5.1 The kubeadm configuration (controlplane)

Blocks labelled `# controlplane` run in a root shell on the controlplane. From here on, keep one PowerShell window connected to it:

```PowerShell
ssh -p 2210 root@127.0.0.1
```

Check: the prompt says `[root@controlplane ~]#`. The same goes for `# worker1` (port 2211) and `# worker2` (port 2212) when a block needs them.

Why each part:

- `advertiseAddress` / `controlPlaneEndpoint` **10.0.1.10:** the API server listens on the public network, not on any other NIC.
- `criSocket` **CRI-O:** the runtime chosen in Phase 2.
- `podSubnet` **10.244.0.0/16:** Flannel's default. The doc's Grid response file names `eth0:10.244.0.0` as the public network inside the pods, so this range has to match.
- `dnsDomain` **cluster.local:** the SCAN name becomes `racnode-scan.rac.svc.cluster.local`.
- `allowedUnsafeSysctls`: the RAC pods set `kernel.shmall`, `kernel.shmmax`, `kernel.shmmni`, `kernel.sem` and some `net.*` settings for themselves. Kubernetes calls these "unsafe" and the kubelet rejects any pod that sets them, unless they're allowed here. These settings are namespaced: they affect only that pod, not the node. The OCNE doc sets this as a kubelet flag; with kubeadm it belongs in the `KubeletConfiguration`, which every node picks up when it joins.

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
cat >/root/kubeadm-config.yaml <<'EOF'
apiVersion: kubeadm.k8s.io/v1beta4
kind: InitConfiguration
localAPIEndpoint:
  advertiseAddress: 10.0.1.10
  bindPort: 6443
nodeRegistration:
  name: controlplane
  criSocket: unix:///var/run/crio/crio.sock
---
apiVersion: kubeadm.k8s.io/v1beta4
kind: ClusterConfiguration
clusterName: rac-poc
controlPlaneEndpoint: 10.0.1.10:6443
networking:
  podSubnet: 10.244.0.0/16
  serviceSubnet: 10.96.0.0/12
  dnsDomain: cluster.local
---
apiVersion: kubelet.config.k8s.io/v1beta1
kind: KubeletConfiguration
cgroupDriver: systemd
allowedUnsafeSysctls:
  - "kernel.shm*"
  - "kernel.sem"
  - "net.*"
EOF
```

5.2 Initialise the control plane (controlplane)

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubeadm init --config /root/kubeadm-config.yaml
mkdir -p ~/.kube && cp -f /etc/kubernetes/admin.conf ~/.kube/config
kubectl get nodes
```

Check: `Your Kubernetes control-plane has initialized successfully!`. The controlplane shows `NotReady` until there's a pod network in 5.3, which is expected.

5.3 Flannel, the pod network (controlplane)

What: Flannel gives every pod an `eth0` address in 10.244.0.0/16, and routes pod traffic between nodes over the 10.0.1.x network.

Why: Kubernetes needs a CNI network before any pod can start. In the RAC pods, `eth0` is the **public** network: the node names, VIPs and SCAN all resolve to `eth0` addresses.

This URL always fetches the newest Flannel release. The lab used the release current on 23 Oct 2026. If a newer one misbehaves, pick a specific version from Flannel's releases page instead of `latest`.

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
kubectl -n kube-flannel rollout status ds/kube-flannel-ds
kubectl get nodes
```

Check: the controlplane turns `Ready` within a minute.

5.4 Join the workers

What: a join command with a fresh token, run on each worker. The PC fetches it from the controlplane and runs it on both workers, so there's nothing to copy by hand. The token in it expires after 24 hours.

PowerShell

```
$join = ssh -p 2210 root@127.0.0.1 "kubeadm token create --print-join-command"; $join
ssh -p 2211 root@127.0.0.1 $join
ssh -p 2212 root@127.0.0.1 $join
```

Each worker ends with `This node has joined the cluster`.

Back on the controlplane, label the workers and wait for them:

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl label node worker1 worker2 node-role.kubernetes.io/worker= --overwrite
kubectl wait --for=condition=Ready node --all --timeout=600s
kubectl get nodes -o wide
kubectl get pods -A
```

Check:

- three nodes, all `Ready` and on `v1.36.x`, with INTERNAL-IP 10.0.1.10, .11 and .12;
- every system pod `Running`: coredns, etcd, kube-apiserver, kube-proxy, kube-flannel.

If a worker shows the wrong INTERNAL-IP, the `--node-ip` setting from Phase 4 is missing.

5.5 kubectl from your PC (optional)

Why: so you can run `kubectl get pods` from Git Bash on Windows instead of SSH-ing into the controlplane first. It's handy in a demo.

PowerShell

```
winget install -e --id Kubernetes.kubectl
& "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe" natnetwork modify --netname k8s-public --port-forward-4 "k8sapi:tcp:[]:6443:[10.0.1.10]:6443"
```

Git Bash

```
mkdir -p ~/.kube
scp -P 2210 root@127.0.0.1:/etc/kubernetes/admin.conf ~/.kube/config
kubectl config set-cluster rac-poc --server=https://127.0.0.1:6443 --tls-server-name=kubernetes
echo 'export MSYS_NO_PATHCONV=1' >> ~/.bashrc
echo "alias kx='winpty kubectl'" >> ~/.bashrc
source ~/.bashrc
kubectl get nodes
```

Why each line:

- `set-cluster rac-poc` : kubeadm names the cluster in `admin.conf` after `clusterName` from 5.1, so the entry to change is `rac-poc` . Changing an entry called `kubernetes` would add an unused one, and kubectl would still try 10.0.1.10.
- `--tls-server-name=kubernetes` : the API server's certificate is issued for `kubernetes` and 10.0.1.10, not for 127.0.0.1. This keeps certificate checking on.
- `MSYS_NO_PATHCONV=1` : stops Git Bash rewriting `/u01/...` into `C:/Program Files/Git/u01/...` in `kubectl exec` commands.
- `kx` **(winpty):** needed for interactive shells, for example `kx exec -it racnode1-0 -- bash` .

`admin.conf` gives full control of the cluster. Keep it out of anything you share.

PHASE 6

Multus and the RAC private networks

Result of this phase: pods can have two extra interfaces, `eth1` on 10.0.11.0/24 and `eth2` on 10.0.12.0/24, with addresses you choose. A test between the workers proves the interconnect works before any Oracle software is involved.

6.1 Why RAC needs Multus

A Kubernetes pod normally has exactly one network interface, `eth0` , from the cluster network (Flannel here). RAC needs more than that:

- **A public network:** client connections, the VIPs and SCAN. That's `eth0` .
- **A private interconnect:** dedicated, low-latency and not routed through the cluster network. Cache Fusion moves database blocks between the instances over it, and the cluster heartbeat runs on it. If the heartbeat is lost, Grid evicts a node.

Multus is a "meta" CNI plugin that attaches additional networks to a pod. Each extra network is described by a **NetworkAttachmentDefinition** (NAD). This lab uses `macvlan` , which gives the pod its own MAC address

directly on the worker's `enp0s8` / `enp0s9`. Two networks give Grid's **HAIP** (highly available IP) two paths, so the interconnect survives losing one.

6.2 Install Multus (controlplane)

The thick-plugin variant runs a small daemon on every node. The URL follows Multus's main branch, as it did in the lab on 23 Oct 2026. For a repeatable build you can replace `master` with a release tag.

`controlplane`

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl apply -f https://raw.githubusercontent.com/k8snetworkplumbingwg/multus-
cni/master/deployments/multus-daemonset-thick.yml
kubectl -n kube-system rollout status ds/kube-multus-ds --timeout=300s
```

Check: `daemon set "kube-multus-ds" successfully rolled out`.

6.3 The two RAC networks (controlplane)

Why each setting:

- `master: enp0s8` / `enp0s9`: the worker NICs prepared in Phase 4. The doc uses `ens5` / `ens6`, its own NIC names.
- `mode: bridge`: pods on the same parent NIC can reach each other.
- `host-local` **IPAM with a range of 140–150:** the doc's subnets and ranges.
- `"capabilities": {"ips": true}`: lets each pod request a **specific** address (10.0.11.141 for racnode1-0, .142 for racnode2-0). Without it, `host-local` ignores the request and picks addresses per node on its own, so both workers can hand out the same IP. The doc's NADs don't have this line; this is a delta.

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl create namespace multus-rac
cat <<'EOF' | kubectl apply -f -
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: macvlan-conf1
  namespace: multus-rac
spec:
  config: '{ "cniVersion": "0.3.1", "type": "macvlan", "master": "enp0s8", "mode": "bridge",
    "capabilities": { "ips": true },
    "ipam": { "type": "host-local", "subnet": "10.0.11.0/24", "rangeStart": "10.0.11.140", "rangeEnd":
"10.0.11.150" } }'
---
apiVersion: k8s.cni.cncf.io/v1
kind: NetworkAttachmentDefinition
metadata:
  name: macvlan-conf2
  namespace: multus-rac
spec:
  config: '{ "cniVersion": "0.3.1", "type": "macvlan", "master": "enp0s9", "mode": "bridge",
    "capabilities": { "ips": true },
    "ipam": { "type": "host-local", "subnet": "10.0.12.0/24", "rangeStart": "10.0.12.140", "rangeEnd":
"10.0.12.150" } }'
EOF
kubectl -n multus-rac get network-attachment-definitions
```

6.4 Prove the interconnect (controlplane)

What: one throwaway pod on each worker, each with `eth1` and `eth2` at chosen addresses. Then pings across.

Why now: if the private network doesn't work, Grid's installer fails much later with a far less obvious error. In the first build, this test is what exposed the promiscuous-mode problem. `NET_RAW` is added because CRI-O drops it by default and `ping` needs it.

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
for n in 1 2; do o=$((147+n)); cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: nettest-w$n
  namespace: multus-rac
  annotations:
    k8s.v1.cni.cncf.io/networks: '[{"name":"macvlan-conf1","interface":"eth1","ips":["10.0.11.$o/24"]},
{"name":"macvlan-conf2","interface":"eth2","ips":["10.0.12.$o/24"]}]'
spec:
  nodeSelector: { kubernetes.io/hostname: worker$n }
  containers:
  - name: net
    image: docker.io/library/busybox:1.36
    command: ["sleep", "infinity"]
    securityContext: { capabilities: { add: ["NET_RAW"] } }
EOF
done
kubectl -n multus-rac wait --for=condition=Ready pod/nettest-w1 pod/nettest-w2 --timeout=300s
kubectl -n multus-rac exec nettest-w1 -- ip -4 addr
kubectl -n multus-rac exec nettest-w1 -- ping -c3 10.0.11.149
kubectl -n multus-rac exec nettest-w1 -- ping -c3 10.0.12.149
kubectl -n multus-rac exec nettest-w2 -- ping -c3 10.0.11.148
kubectl -n multus-rac exec nettest-w2 -- ping -c3 10.0.12.148
```

Check:

- `nettest-w1` has `eth1` 10.0.11.148/24 and `eth2` 10.0.12.148/24;
- all four pings get 0% packet loss.

If you get 100% loss, check that `enp0s8` / `enp0s9` show `PROMISC` on both workers (Phase 4, step c on each worker), and that the VirtualBox adapters are set to Allow All (Phase 1.1).

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl -n multus-rac delete pod nettest-w1 nettest-w2
```

PHASE 7

ASM disks as Kubernetes block volumes

Result of this phase: the `rac` namespace, plus two PersistentVolumes and two PersistentVolumeClaims that hand the shared disks to the RAC pods as **raw block devices**.

7.1 Why block PVs, and why "by-id"

- **ASM wants raw disks.** It manages the disks itself, with no filesystem. In Kubernetes that's a PersistentVolume with `volumeMode: Block`: the pod sees a device such as `/dev/asm-disk1`, not a mounted directory.
- `local` **PVs with node affinity for worker1 and worker2:** the same physical disk is visible on both workers, so the PV may be used on either.
- `ReadWriteMany`**:** both RAC pods use the same disks at the same time. ASM coordinates access between the instances.
- `persistentVolumeReclaimPolicy: Retain`**:** deleting a claim never wipes the database disks.
- `/dev/disk/by-id/...` **instead of** `/dev/sdd`**:** the doc uses `/dev/sdd` and `/dev/sde`, but `sdX` names can differ between the workers and change between boots (Phase 4). The by-id name comes from the disk's serial number, which is the same on both workers because it's the same VDI file. This is a delta from the doc.

7.2 Find the disk IDs (from the PC)

What: inside the workers, each shared disk appears as `/dev/disk/by-id/ata-VBOX_HARDDISK_VB<first 8 characters of the VDI's UUID>-...`. This block reads the two UUIDs from VirtualBox, finds the matching names on worker1 and worker2, and checks that both workers see the same two names.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"; $Base = "D:\VMs\rac-poc"
$ASM = @{}
foreach ($d in "asm1","asm2") {
  $pre = ((& $VB showmediuminfo disk "$Base\shared\$d.vdi") | Select-String '^UUID:\s+([0-9a-f]
{8})').Matches[0].Groups[1].Value
  $w1 = ssh -p 2211 root@127.0.0.1 "ls /dev/disk/by-id/ | grep -v part | grep ^ata-VBOX_HARDDISK_VB$pre-"
  $w2 = ssh -p 2212 root@127.0.0.1 "ls /dev/disk/by-id/ | grep -v part | grep ^ata-VBOX_HARDDISK_VB$pre-"
  if ($w1 -and ($w1 -eq $w2) -and (@($w1).Count -eq 1)) { $ASM[$d] = $w1; "OK    $d = $w1 (same on
worker1 and worker2)" }
  else { "STOP  $d (VB$pre): worker1 sees '$w1', worker2 sees '$w2'" }
}
```

Check: two `OK` lines, one for asm1 and one for asm2. Keep this PowerShell window open: 7.3 uses `$ASM` from it. If a line says STOP, don't go on; paste the output.

7.3 Create the namespace, PVs and PVCs (from the PC, run on controlplane)

What: the same PowerShell window sends the definitions to the controlplane, with the two IDs from 7.2 filled in.

PowerShell (same window as 7.2)

```
if ($ASM.asm1 -and $ASM.asm2) {
$script = "ASM1=$($ASM.asm1); ASM2=$($ASM.asm2)`n" + @'
kubectl create namespace rac
for i in 1 2; do id=$([ $i = 1 ] && echo $ASM1 || echo $ASM2); cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolume
metadata:
  name: block-asm-disk1-pv-vol$i
  labels: { asm_vol: block-asm-pv-vol$i }
spec:
  volumeMode: Block
  capacity: { storage: 50Gi }
  accessModes: [ReadWriteMany]
  persistentVolumeReclaimPolicy: Retain
  local: { path: /dev/disk/by-id/$id }
  nodeAffinity:
    required:
      nodeSelectorTerms:
      - matchExpressions:
        - { key: kubernetes.io/hostname, operator: In, values: [worker1, worker2] }
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: block-asm-pvc-vol$i
  namespace: rac
spec:
  accessModes: [ReadWriteMany]
  volumeMode: Block
  storageClassName: ""
  resources: { requests: { storage: 50Gi } }
  selector: { matchLabels: { asm_vol: block-asm-pv-vol$i } }
EOF
done
kubectl get pv; kubectl -n rac get pvc
'@
$script | ssh -p 2210 root@127.0.0.1 "tr -d '\r' | bash -s"
} else { "STOP: run 7.2 in this window first; it must print two OK lines" }
```

Check: both PVs and both PVCs are `Bound`, each PVC to its own PV. The label selector makes sure claim 1 gets disk 1.

Note: don't put anything on these disks. The validation in the first build checks that they're blank before the Grid install; afterwards they carry ASM's own `oracleasm` signature.

PHASE 8

The RAC container image

Result of this phase: the image `localhost/oracle/database-rac:19-ol9-slim` is in the local image store on both workers. It's built from the doc's recipe and contains **no Oracle software**.

8.1 What's in the image, and what isn't

The image is a small Oracle Linux 9 system that behaves like a server:

- **systemd as PID 1:** Grid Infrastructure is written for a full OS. It installs systemd services (`oracle-ohasd`), uses `rc.local` , and expects `sshd` . So the pod runs `/usr/sbin/init` rather than one application process.
- `oracle-database-preinstall-19c` **:** creates the `oracle` user and the `dba` groups, and sets the limits and packages the database needs.
- **The ASM users and groups:** `grid` (UID 54332) and `asmadmin` / `asmdba` / `asmoper` , with **fixed IDs**. The Oracle homes live on `/u01` on the worker and survive the pod, so a recreated pod must have exactly the same UIDs, or every file would suddenly belong to a stranger.
- `setupContainerEnv.sh` **, run from** `rc.local` **at every start:** gives `/dev/asm-disk1` and `/dev/asm-disk2` to `grid:asmadmin` with mode 660. Kubernetes creates block devices as root, and ASM must be able to open them. It also calls a restart hook on `/u01` (Phase 14). That one line is this lab's addition.
- **Not in the image:** the Grid and database software. They're installed into `/u01` , which is a host directory, so an image rebuild never touches an installed database.

The image differs from the doc's in these small ways:

- the full registry name for the base image;
- the `dnf-makecache` timer removed as well;
- `rc.local` created if it's missing;
- the NFS `rpc_pipefs` mount masked. It can't mount in a container and leaves systemd "degraded", which Grid's checks flag.

Oracle's GitHub RAC image was *not* used: its `rc.local` starts its own automation on every boot.

8.2 Write the build files (worker1)

worker1

CONNECT FIRST `ssh -p 2211 root@127.0.0.1`

```
cd /scratch/image
cat >setupContainerEnv.sh <<'EOF'
#!/bin/bash
for d in /dev/asm-disk*; do
  [ -b "$d" ] || continue
  chown grid:asmadmin "$d"
  chmod 660 "$d"
done
systemctl disable --now dnf-makecache.timer dnf-makecache.service >/dev/null 2>&1
systemctl reset-failed
[ -x /u01/app/restart-state/onstart.sh ] && /u01/app/restart-state/onstart.sh
exit 0
EOF
chmod +x setupContainerEnv.sh
cat >Containerfile <<'EOF'
FROM container-registry.oracle.com/os/oraclelinux:9
ENV SCRIPT_DIR=/opt/scripts/startup \
    SETUPCONTAINERENV=setupContainerEnv.sh
COPY $SETUPCONTAINERENV $SCRIPT_DIR/
RUN set -e; \
    dnf install -y oracle-database-preinstall-19c systemd vim openssh-server hostname \
                   policycoreutils-python-utils libxcrypt-compat; \
    dnf clean all; \
    groupadd -g 54334 asmadmin; \
    groupadd -g 54335 asmdba; \
    groupadd -g 54336 asmoper; \
    useradd -u 54332 -g oinstall -G oinstall,asmadmin,asmdba,asmoper,racdba grid; \
    usermod -g oinstall -G oinstall,dba,oper,backupdba,dgdba,kmdba,asmdba,racdba oracle; \
    sed 's/oracle/grid/g' /etc/security/limits.d/oracle-database-preinstall-19c.conf \
        > /etc/security/limits.d/grid-database-preinstall-19c.conf; \
    rm -f /etc/systemd/system/oracle-database-preinstall-19c-firstboot.service; \
    sed -i 's/^TasksMax\S*/TasksMax=80%/g' /usr/lib/systemd/system/user-.slice.d/10-defaults.conf; \
    rm -f /usr/lib/systemd/system/dnf-makecache.service /usr/lib/systemd/system/dnf-makecache.timer; \
    if [ ! -f /etc/rc.d/rc.local ]; then mkdir -p /etc/rc.d; printf '#!/bin/bash\n' > /etc/rc.d/rc.local;
fi; \
    [ -e /etc/rc.local ] || ln -s rc.d/rc.local /etc/rc.local; \
    echo "$SCRIPT_DIR/$SETUPCONTAINERENV" >> /etc/rc.d/rc.local; \
    chmod +x /etc/rc.d/rc.local $SCRIPT_DIR/$SETUPCONTAINERENV; \
    setcap 'cap_net_admin,cap_net_raw+ep' /usr/bin/ping; \
    systemctl mask var-lib-nfs-rpc_pipefs.mount; \
    sync
USER root
WORKDIR /root
CMD ["/usr/sbin/init"]
EOF
```

A few lines that are easy to misread:

- `TasksMax=80%` **:** systemd's default cap on processes per user is too low for Grid and the database, which start hundreds of processes.
- **The grid limits file:** a copy of the oracle user's limits (open files, processes, memlock), because the preinstall package only covers `oracle`.
- `setcap` **on ping:** Grid's checks ping as a non-root user.

8.3 Build (worker1)

Why build on worker1: podman and CRI-O share `/var/lib/containers`, so the image is usable by Kubernetes the moment the build finishes. The doc's StatefulSets refer to it as `localhost/...`, which is never pulled from a registry.

worker1

CONNECT FIRST `ssh -p 2211 root@127.0.0.1`

```
cd /scratch/image
podman build -t localhost/oracle/database-rac:19-ol9-slim -f Containerfile .
crictl images | grep database-rac
```

The build takes 3 to 6 minutes, mostly `dnf install`. Pulling the base image from `container-registry.oracle.com` needs no login.

8.4 Copy the image to worker2

Why worker to worker: the first attempt copied through the PC and the VirtualBox port forward, and managed 34 MB in five minutes. Directly between the workers on 10.0.1.x, the 400 MB image takes seconds. worker2 first needs to trust worker1's SSH key; the PC passes the key across.

PowerShell

```
ssh -p 2211 root@127.0.0.1 "test -f ~/.ssh/id_ed25519 || ssh-keygen -q -t ed25519 -N '' -f
~/.ssh/id_ed25519; cat ~/.ssh/id_ed25519.pub" | ssh -p 2212 root@127.0.0.1 "cat >>
~/.ssh/authorized_keys"
```

worker1

CONNECT FIRST `ssh -p 2211 root@127.0.0.1`

```
podman image save -o /var/tmp/database-rac.tar localhost/oracle/database-rac:19-ol9-slim
scp -o StrictHostKeyChecking=accept-new /var/tmp/database-rac.tar root@10.0.1.12:/var/tmp/
ssh root@10.0.1.12 'podman image load -i /var/tmp/database-rac.tar && rm -f /var/tmp/database-rac.tar &&
crictl images | grep database-rac'
rm -f /var/tmp/database-rac.tar
```

Check: `crictl images` on **both** workers lists `localhost/oracle/database-rac 19-ol9-slim` with the **same image ID**.

worker1 (quick look inside the image)

CONNECT FIRST `ssh -p 2211 root@127.0.0.1`

```
podman run --rm --entrypoint /bin/bash localhost/oracle/database-rac:19-ol9-slim -c 'id grid; id oracle;
cat /etc/rc.d/rc.local; getcap /usr/bin/ping'
```

You should see grid as `uid=54332` in `asmadmin`, oracle in `asmdba`, `rc.local` calling `setupContainerEnv.sh`, and `ping` with `cap_net_admin,cap_net_raw`.

PHASE 9

Stage the Oracle software

Result of this phase: `oracle-gridhome-19.32.zip` and `oracle-dbhome-19.32.zip` are on worker1 in `/scratch/software/stage`. The RAC pod `racnode1-0` will see them as `/stage`.

9.1 Why gold images, and why only worker1

- **Gold images:** these zips are Oracle homes that already have the 19.32 Release Update and the OJVM update applied. That replaces the doc's base 19.3 software, plus `-applyRU`, plus the OPatch swap. Fewer steps, and the same patch level on every install. The SQL part of the patches is applied to the database later by `datapatch`, which DBCA runs by itself (Phase 12).
- **Only worker1:** the installers run on racnode1-0 and copy the software to racnode2-0 themselves, over SSH between the pods.

9.2 Serve the zips from the PC (Git Bash)

Why not push them through the SSH port forward: 9.6 GB through VirtualBox's NAT forwarding ran at about 0.1 MB/s. The VirtualBox NAT host address (10.0.1.2) didn't answer from the VMs, so the worker downloads from the PC's **LAN address** instead, at 40–125 MB/s.

Git Bash (leave this window open while worker1 downloads)

```
mkdir -p ~/Downloads/oracle-stage
mv ~/Downloads/oracle-*-19.32.zip ~/Downloads/oracle-stage/
cd ~/Downloads/oracle-stage && python -m http.server 8000 --bind 0.0.0.0
```

Only the two zips are in that folder, so only they are served. If Windows Firewall asks, allow Python on private networks.

9.3 Download on worker1

PowerShell (finds the PC's LAN address, 192.168.1.160 in this lab)

```
$ip = (Get-NetIPConfiguration | Where-Object IPv4DefaultGateway).IPv4Address.IPAddress | Select-Object -First 1; $ip
ssh -p 2211 root@127.0.0.1 "cd /scratch/software/stage && for f in oracle-gridhome-19.32.zip oracle-dbhome-19.32.zip; do curl -fSL -o `$f http://${ip}:8000/`$f || exit 1; done; ls -lh; sha256sum oracle-*-19.32.zip"
```

PowerShell (the same checksums on the PC)

```
Get-FileHash "$env:USERPROFILE\Downloads\oracle-stage\oracle-*-19.32.zip" -Algorithm SHA256 | Format-List Hash, Path
```

Check: both files are on worker1 with the same sizes, and the SHA-256 values match the PC's (PowerShell prints them in capitals). Then stop the server with Ctrl+C.

PHASE 10

RAC pods: SSH secret, headless services, StatefulSets

Result of this phase: two RAC pods, `racnode1-0` on worker1 and `racnode2-0` on worker2. Each has systemd running, `eth0` on the pod network, `eth1` / `eth2` on the private networks, both ASM disks, `/u01` on its worker's disk, and DNS names for itself, its VIP and the SCAN. They're ready for the Grid install.

10.1 How each Kubernetes object maps to a RAC need

RAC NEEDS	KUBERNETES OBJECT	WHY THAT OBJECT
Two servers with stable names	**StatefulSet** per node (`racnode1`, `racnode2`)	Gives a predictable pod name (`racnode1-0`) that becomes the host name Grid registers. A Deployment would give a random name on every restart.
Each node always on the same host	**nodeAffinity** (`worker1` / `worker2`)	`/u01` is a directory on that worker, and Grid's local registry lives in it. The pod can never move to another node.
Host names, VIPs and SCAN in DNS	**Headless Services** (`clusterIP: None`)	CoreDNS answers with the **pod IP itself**, not a load-balanced service IP. Grid needs real addresses that it can bring up and check.
Names that resolve before the pod is "Ready"	`publishNotReadyAddresses: true`	The pods are only Ready once port 1521 answers (after Grid). Grid's installer needs the names first, so without this, it would never be installed.

RAC NEEDS	KUBERNETES OBJECT	WHY THAT OBJECT
SCAN spread over both nodes	Headless Service `racnode-scan`, selector `cluster=racnode`	Matches both pods, so the SCAN name returns both IPs.
Private interconnect	Multus annotation with fixed `ips`	`eth1` and `eth2` at .141 and .142 on both private networks (Phase 6).
Shared ASM disks	PVCs with `volumeDevices`	Raw devices at `/dev/asm-disk1` and `/dev/asm-disk2`, the same disks in both pods (Phase 7).
Oracle homes that outlive the pod	`hostPath` volume on `/u01`	The software and the node's local configuration survive pod deletion.
Large shared memory	`emptyDir` with `medium: Memory` on `/dev/shm`, plus `hugepages-2Mi`	Default container `/dev/shm` is 64 MB. The SGA goes into hugepages.
Kernel IPC settings for Oracle	Pod `securityContext.sysctls`	`kernel.shm*` and `kernel.sem` per pod. They must be allowed in the kubelet config (Phase 5).
Grid managing IPs and priorities	Capabilities `NET_ADMIN`, `NET_RAW`, `SYS_NICE`, `SYS_RESOURCE` ...	Grid brings VIPs up and down, runs real-time cluster processes and raises limits, without the pod being fully privileged.
Clean shutdown	`preStop: kill -SIGRTMIN+3 1`	Tells systemd in the pod to shut down in an orderly way when the pod is deleted.
Passwordless SSH between nodes (installer)	**Secret** `ssh-key-secret`, mounted read-only at `/mnt/.ssh`	One key pair shared by both pods. The installers copy the software between nodes over SSH.
"Is this node usable?"	`readinessProbe` on TCP 1521	Ready only when the listener is up. That's why the pods show `0/1` until Grid is installed.

10.2 The SSH key secret (controlplane)

Why a fixed key pair: the Grid and database installers on racnode1-0 copy files to racnode2-0 and run commands there over SSH, as `grid` and as `oracle`. Both pods get the same key from the Secret, so each trusts the other from the first boot.

`controlplane`

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
mkdir -p /root/rac && cd /root/rac
ssh-keygen -q -t rsa -b 3072 -N "" -C "Key_Pair_for_ssh_connectivity_for_rac_setup" -f /root/rac/id_rsa
kubectl -n rac create secret generic ssh-key-secret --from-file=/root/rac/id_rsa --from-
file=/root/rac/id_rsa.pub
```

10.3 The five headless services (controlplane)

What: `racnode1-0` and `racnode2-0` (the hosts), `racnode1-0-vip` and `racnode2-0-vip` (the VIPs), and `racnode-scan`.

Why the VIP services point at the pod itself: on bare metal a VIP is a separate address that moves to the surviving node. Here each VIP name resolves to its pod's own `eth0` address. Flannel routes each node's pod range only to that worker, so an address can't move between workers anyway. Clients survive a node loss through SCAN instead (Phase 13).

`controlplane`

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
for s in racnode1-0:racnode1-0 racnode1-0-vip:racnode1-0 racnode2-0:racnode2-0 racnode2-0-vip:racnode2-0;
do
  name=${s%%:*}; pod=${s##*:}; cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Service
metadata:
  name: $name
  namespace: rac
  labels: { statefulset.kubernetes.io/pod-name: $pod }
spec:
  clusterIP: None
  type: ClusterIP
  publishNotReadyAddresses: true
  selector: { statefulset.kubernetes.io/pod-name: $pod }
EOF
done
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Service
metadata:
  name: racnode-scan
  namespace: rac
  labels: { cluster: racnode }
spec:
  clusterIP: None
  type: ClusterIP
  publishNotReadyAddresses: true
  selector: { cluster: racnode }
EOF
kubectl -n rac get svc
```

10.4 The two StatefulSets (controlplane)

The doc's StatefulSets, with these changes:

- **Placement:** pinned to `worker1` / `worker2`.
- **Image:** this lab's image, with `imagePullPolicy: IfNotPresent`. The doc's `Always` tries to pull a local-only image from a registry and fails.
- **Resources:** 12 GiB memory, 3 CPUs and 4 GiB of hugepages per pod, instead of 16 GiB, 4 and 8 GiB, to fit the 22 GB, 4-vCPU workers.

- `/mnt/.ssh` **read-only**, as the doc's text asks.
- **Selector:** each selector also matches `hostname: racnode<n>-0`. The doc's two StatefulSets share the selector `cluster=racnode`, which overlaps, and Kubernetes warns against that.

`/stage` (the zips) is only mounted on node 1, where the installers run.

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
for n in 1 2; do
  stagevol=""; stagemnt=""
  if [ $n = 1 ]; then
    stagevol=$'       - name: stagevol\n         hostPath: { path: /scratch/software/stage }'
    stagemnt=$'         - { mountPath: /stage, name: stagevol }'
  fi
  cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: racnode$n
  namespace: rac
  labels: { cluster: racnode, hostname: racnode$n-0 }
spec:
  selector:
    matchLabels: { cluster: racnode, hostname: racnode$n-0 }
  serviceName: racnode
  template:
    metadata:
      labels: { cluster: racnode, hostname: racnode$n-0 }
      annotations:
        k8s.v1.cni.cncf.io/networks: '[
          {"name":"macvlan-conf1","namespace":"multus-rac","interface":"eth1","ips":["10.0.11.14$n/24"]},
          {"name":"macvlan-conf2","namespace":"multus-rac","interface":"eth2","ips":["10.0.12.14$n/24"]}
]'
    spec:
     securityContext:
       sysctls:
        - { name: kernel.shmall, value: "2097152" }
        - { name: kernel.sem, value: "250 32000 100 128" }
        - { name: kernel.shmmax, value: "8589934592" }
        - { name: kernel.shmmni, value: "4096" }
        - { name: net.ipv4.conf.all.rp_filter, value: "2" }
     affinity:
       nodeAffinity:
         requiredDuringSchedulingIgnoredDuringExecution:
           nodeSelectorTerms:
           - matchExpressions:
             - { key: kubernetes.io/hostname, operator: In, values: [worker$n] }
     volumes:
       - name: myshm
         emptyDir: { medium: Memory }
       - name: swvol-racnode$n
         hostPath: { path: /scratch/rac/cluster01/node$n }
       - name: asmvol1
         persistentVolumeClaim: { claimName: block-asm-pvc-vol1 }
       - name: asmvol2
         persistentVolumeClaim: { claimName: block-asm-pvc-vol2 }
       - name: sshkeys
```

```
        secret: { secretName: ssh-key-secret }
$stagevol
    containers:
      - name: racnode$n-0
        image: localhost/oracle/database-rac:19-ol9-slim
        imagePullPolicy: IfNotPresent
        command: [/usr/sbin/init]
        resources:
          requests: { memory: "12Gi", cpu: "3" }
          limits: { hugepages-2Mi: "4Gi", memory: "12Gi", cpu: "3" }
        securityContext:
          privileged: false
          capabilities:
            add: ["NET_ADMIN", "NET_RAW", "SYS_CHROOT", "SYS_NICE", "SYS_RESOURCE", "AUDIT_WRITE",
"AUDIT_CONTROL"]
        lifecycle:
          preStop: { exec: { command: ["/bin/bash","-c","kill -SIGRTMIN+3 1"] } }
        volumeDevices:
        - { name: asmvol1, devicePath: /dev/asm-disk1 }
        - { name: asmvol2, devicePath: /dev/asm-disk2 }
        readinessProbe:
          tcpSocket: { port: 1521 }
          initialDelaySeconds: 10
          periodSeconds: 30
          failureThreshold: 1
        volumeMounts:
        - { mountPath: /dev/shm, name: myshm }
        - { mountPath: /u01, name: swvol-racnode$n }
        - { mountPath: /mnt/.ssh, name: sshkeys, readOnly: true }
$stagemnt
EOF
done
kubectl -n rac get pods -o wide -w
```

Press Ctrl+C when both pods show `Running` with `0/1` READY. `0/1` is correct until Grid opens port 1521.

If a pod restarts every second with no log output, `container_manage_cgroup` is off on that worker (Phase 2.6). That's the first-build failure: `setsebool -P container_manage_cgroup on`, then delete the pod.

10.5 Check each pod (controlplane)

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
for n in 1 2; do p=racnode$n-0; peer=$((3-n)); echo "--- $p"
  kubectl -n rac exec $p -- bash -c "
    systemctl is-system-running; hostname
    ip -4 -br addr show eth1; ip -4 -br addr show eth2
    ping -c2 -W2 10.0.11.14$peer >/dev/null && echo 'ping eth1 peer OK'
    ping -c2 -W2 10.0.12.14$peer >/dev/null && echo 'ping eth2 peer OK'
    stat -c '%n %U:%G %a' /dev/asm-disk1 /dev/asm-disk2
    touch /u01/.w && rm /u01/.w && echo '/u01 writable'
    touch /mnt/.ssh/x 2>/dev/null || echo '/mnt/.ssh read-only'
    for h in racnode1-0 racnode2-0 racnode1-0-vip racnode2-0-vip racnode-scan; do echo \"\$h -> \$(getent
ahostsv4 \$h | awk '{print \$1}' | sort -u | xargs)\"; done
    ls /stage 2>/dev/null"
done
```

Check, for each pod:

- systemd says `running` or `degraded`, and the host name is the pod name;
- `eth1` and `eth2` are .141 on racnode1-0 and .142 on racnode2-0, and both pings are OK;
- `/dev/asm-disk1` and `/dev/asm-disk2` are `grid:asmadmin 660`;
- `/u01` is writable, and `/mnt/.ssh` is read-only;
- all five names resolve, and `racnode-scan` returns **both** pod IPs;
- on racnode1-0, `/stage` lists both zips.

PHASE 11

Grid Infrastructure in the pods

Result of this phase: Oracle Grid Infrastructure 19.32 runs in both pods:

- the clusterware stack on both nodes;
- ASM with the `DATA` disk group on the two shared disks;
- the node VIPs, the SCAN VIPs and the SCAN listeners;
- HAIP on the two private networks.

It takes about 45–75 minutes, mostly installer and `root.sh` time.

11.0 Working inside a pod

Every block in this phase runs **as root inside a RAC pod**. Open a shell from the controlplane:

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl -n rac exec -it racnode1-0 -- bash
```

Long steps run detached, with a log. A `kubectl exec` session can drop: a network blip, the PC sleeping, or an SSH timeout. If it drops while an installer runs in the foreground, the installer dies with it. So each long step runs detached (`setsid`) into `/u01/poc-logs/<step>.log` , and you follow the log with `tail -f` . Ctrl+C stops the `tail` , not the step.

11.1 Users, SSH keys, directories and profiles (both pods)

Why:

- **The installers SSH between the nodes as `grid` and `oracle` .** Both users get the key pair from the Secret.
- **The directory tree** is created empty and owned by `grid` (the Grid home and inventory) and `oracle` (the database base), as the doc does.
- **Each user's `.bashrc`** sets `ORACLE_HOME` and `PATH` .

Set ownership recursively only while the homes are empty. In the first build, rerunning this step after `root.sh` ran `chown -R` / `chmod -R` over an installed Grid home. That stripped the setuid bits and the root ownership `root.sh` had set. The result was `ORA-12547` for the `oracle` user, and DBCA could no longer see the ASM disk groups (DBT-05801). The guard below refuses once a home has content.

racnode1-0, then the same in racnode2-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash` ; for racnode2-0: `kubectl -n rac exec -it racnode2-0 -- bash`

```
for u in grid oracle; do
  h=/home/$u; mkdir -p $h/.ssh; chmod 700 $h/.ssh
  cat /mnt/.ssh/id_rsa > $h/.ssh/id_rsa; cat /mnt/.ssh/id_rsa.pub > $h/.ssh/id_rsa.pub
  cat /mnt/.ssh/id_rsa.pub > $h/.ssh/authorized_keys
  chmod 400 $h/.ssh/id_rsa; chmod 600 $h/.ssh/authorized_keys; chown -R $u:oinstall $h/.ssh
done
mkdir -p /u01/app/19c/grid /u01/app/grid /u01/app/oraInventory /u01/app/oracle/product/19c/dbhome_1
/u01/poc-logs
if [ -z "$(ls -A /u01/app/19c/grid)" ] && [ -z "$(ls -A /u01/app/oracle/product/19c/dbhome_1)" ]; then
  chown -R grid:oinstall /u01/app; chown -R oracle:oinstall /u01/app/oracle; chmod -R 775 /u01/app
else echo "homes already installed: ownership left alone"; fi
chown grid:oinstall /u01/poc-logs; chmod 775 /u01/poc-logs
[ -d /stage ] && chown grid:oinstall /stage
printf '%s\n' 'export TMPDIR=/var/tmp' 'export ORACLE_HOME=/u01/app/19c/grid' 'export
PATH=$ORACLE_HOME/bin:$PATH' >> /home/grid/.bashrc
printf '%s\n' 'export TMPDIR=/var/tmp' 'export ORACLE_HOME=/u01/app/oracle/product/19c/dbhome_1' 'export
PATH=$ORACLE_HOME/bin:$PATH' >> /home/oracle/.bashrc
```

Once **both** pods have their keys, record each other's host keys and test SSH:

racnode1-0, then the same in racnode2-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash` ; for racnode2-0: `kubectl -n rac exec -it racnode2-0 -- bash`

```
for u in grid oracle; do
  su - $u -c "rm -f ~/.ssh/known_hosts; ssh-keyscan -H racnode1-0 racnode2-0 >> ~/.ssh/known_hosts
2>/dev/null; ssh-keyscan racnode1-0 racnode2-0 >> ~/.ssh/known_hosts 2>/dev/null"
  for t in racnode1-0 racnode2-0; do echo "$u -> $t: $(su - $u -c "ssh -o BatchMode=yes -o
ConnectTimeout=5 $u@$t echo ok" 2>&1)"; done
done
```

Check: 4 lines of `ok` in each pod.

Why both hashed and plain entries: the doc's `ssh-keyscan -H` writes hashed host names, which SSH reads fine. Oracle's verification tool (`cluvfy`) doesn't, and reports `PRVH-1526` / `PRVF-4009` . It then skips that node in its checks.

11.2 Unzip the Grid gold image (racnode1-0)

Why `unzip -K` **:** plain `unzip` silently drops setuid and setgid bits; `-K` keeps any that the zip stores. **This gold image stores none:** its `bin/oracle` is an empty placeholder (0 bytes). The installer links the real `bin/oracle` in 11.4, and `root.sh` sets the owners and setuid bits in 11.5. So right after the unzip the setuid/setgid count is **0**, and that's correct. The check that matters comes after 11.5. (An earlier version of this guide expected a count above 0 here; that was wrong.)

Why the two edits (both from the doc):

- `CV_DESTLOC` **:** points the verification utility's work area at `/var/tmp` .
- `SWAP_SPACE` **commented out:** pods have no swap, and the installer would otherwise refuse.

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
GH=/u01/app/19c/grid
su - grid -c "unzip -K -q -o /stage/oracle-gridhome-19.32.zip -d $GH"
grep -q '^CV_DESTLOC=' $GH/cv/admin/cvu_config || echo 'CV_DESTLOC=/var/tmp' >> $GH/cv/admin/cvu_config
sed -i 's/^SWAP_SPACE=/#SWAP_SPACE=/' $GH/oui/oraparam.ini
chown grid:oinstall $GH/cv/admin/cvu_config $GH/oui/oraparam.ini
du -sh $GH; ls -l $GH/bin/oracle; ls $GH/gridSetup.sh
```

Check: the size is about 11 GB (the 19.32 gold image includes `.patch_storage`), `bin/oracle` is 0 bytes, and `gridSetup.sh` is listed.

11.3 The response file (racnode1-0)

What: `gi.rsp` answers every installer question. The values are the doc's:

- a standalone cluster called `racnode-c` ;
- the nodes `racnode1-0` and `racnode2-0` , with their `-vip` names;
- the SCAN `racnode-scan` on port 1521;
- `DATA` with external redundancy and a 4 MB allocation unit, on `/dev/asm-disk1` and `/dev/asm-disk2` ;

- no GIMR, AFD or GNS.

`INVENTORY_LOCATION` is added.

Why `networkInterfaceList=eth0:10.244.0.0:1,eth1:10.0.11.0:5,eth2:10.0.12.0:5` **:** this is where Kubernetes meets Grid. Type `1` marks the **public** network, which is the Flannel pod network on `eth0`. Type `5` marks **ASM and private** (interconnect plus ASM traffic) on the two Multus networks.

The netmasks are not in this file. `netmaskList` isn't a valid response-file variable (`INS-10105`), so it goes on the command line, as in the doc.

The password is read at a prompt, written only into this file (mode 600), and blanked after the install. **Rules:** 8–30 characters, starting with a letter, letters and digits only, with at least one upper-case letter, one lower-case letter and one digit. A special character got a password rejected in the first build.

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
cat >/u01/poc-logs/gi.rsp <<'EOF'
oracle.install.responseFileVersion=/oracle/install/rspfmt_crsinstall_response_schema_v19.0.0
INVENTORY_LOCATION=/u01/app/oraInventory
oracle.install.option=CRS_CONFIG
ORACLE_BASE=/u01/app/grid
oracle.install.asm.OSDBA=asmdba
oracle.install.asm.OSOPER=asmoper
oracle.install.asm.OSASM=asmadmin
oracle.install.crs.config.scanType=LOCAL_SCAN
oracle.install.crs.config.SCANClientDataFile=
oracle.install.crs.config.gpnp.scanName=racnode-scan
oracle.install.crs.config.gpnp.scanPort=1521
oracle.install.crs.config.ClusterConfiguration=STANDALONE
oracle.install.crs.config.configureAsExtendedCluster=false
oracle.install.crs.config.memberClusterManifestFile=
oracle.install.crs.config.clusterName=racnode-c
oracle.install.crs.config.gpnp.configureGNS=false
oracle.install.crs.config.autoConfigureClusterNodeVIP=false
oracle.install.crs.config.gpnp.gnsOption=
oracle.install.crs.config.gpnp.gnsClientDataFile=
oracle.install.crs.config.gpnp.gnsSubDomain=
oracle.install.crs.config.gpnp.gnsVIPAddress=
oracle.install.crs.config.sites=
oracle.install.crs.config.clusterNodes=racnode1-0:racnode1-0-vip:HUB,racnode2-0:racnode2-0-vip:HUB
oracle.install.crs.config.networkInterfaceList=eth0:10.244.0.0:1,eth1:10.0.11.0:5,eth2:10.0.12.0:5
oracle.install.crs.configureGIMR=false
oracle.install.asm.configureGIMRDataDG=
oracle.install.crs.config.storageOption=
oracle.install.crs.config.sharedFileSystemStorage.votingDiskLocations=
oracle.install.crs.config.sharedFileSystemStorage.ocrLocations=
oracle.install.crs.config.useIPMI=
oracle.install.crs.config.ipmi.bmcUsername=
oracle.install.crs.config.ipmi.bmcPassword=
oracle.install.asm.SYSASMPassword=__PASSWORD__
oracle.install.asm.diskGroup.name=DATA
oracle.install.asm.diskGroup.redundancy=EXTERNAL
oracle.install.asm.diskGroup.AUSize=4
oracle.install.asm.diskGroup.FailureGroups=
oracle.install.asm.diskGroup.disksWithFailureGroupNames=/dev/asm-disk1,,/dev/asm-disk2,
oracle.install.asm.diskGroup.disks=/dev/asm-disk1,/dev/asm-disk2
oracle.install.asm.diskGroup.quorumFailureGroupNames=
oracle.install.asm.diskGroup.diskDiscoveryString=/dev/asm*
oracle.install.asm.monitorPassword=__PASSWORD__
oracle.install.asm.gimrDG.name=
oracle.install.asm.gimrDG.redundancy=
oracle.install.asm.gimrDG.AUSize=1
oracle.install.asm.gimrDG.FailureGroups=
oracle.install.asm.gimrDG.disksWithFailureGroupNames=
oracle.install.asm.gimrDG.disks=
```

```
oracle.install.asm.gimrDG.quorumFailureGroupNames=
oracle.install.asm.configureAFD=false
oracle.install.crs.configureRHPS=false
oracle.install.crs.config.ignoreDownNodes=false
oracle.install.config.managementOption=NONE
oracle.install.config.omsHost=
oracle.install.config.omsPort=0
oracle.install.config.emAdminUser=
oracle.install.config.emAdminPassword=
oracle.install.crs.rootconfig.executeRootScript=false
oracle.install.crs.rootconfig.configMethod=ROOT
oracle.install.crs.rootconfig.sudoPath=
oracle.install.crs.rootconfig.sudoUserName=
oracle.install.crs.config.batchinfo=
oracle.install.crs.app.applicationAddress=
oracle.install.crs.deleteNode.nodes=
EOF
read -rsp "SYSASM / ASMSNMP password: " PW; echo
sed -i "s/__PASSWORD__/$PW/" /u01/poc-logs/gi.rsp; unset PW
chown grid:oinstall /u01/poc-logs/gi.rsp; chmod 600 /u01/poc-logs/gi.rsp
grep -c '=' /u01/poc-logs/gi.rsp
```

11.4 Install the software (racnode1-0, 15–25 minutes)

What: `gridSetup.sh -silent` installs the Grid home on racnode1-0 and copies it to racnode2-0 over SSH.

Exit codes: `0` is success, and `6` is success with warnings. In this lab the warnings were the optional prerequisite checks that a container trips (`INS-13014`). Anything else is a failure: read the log.

`racnode1-0`

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
setsid bash -c 'su - grid -c "/u01/app/19c/grid/gridSetup.sh -silent -waitforcompletion -responseFile
/u01/poc-logs/gi.rsp
oracle.install.crs.config.netmaskList=eth0:255.255.0.0,eth1:255.255.255.0,eth2:255.255.255.0"; echo "exit
$?"' > /u01/poc-logs/gridsetup.log 2>&1 < /dev/null &
tail -f /u01/poc-logs/gridsetup.log
```

Wait for `Successfully Setup Software` and an `exit 0` or `exit 6` line, then press Ctrl+C.

11.5 Root scripts

Why the order matters:

- `orainstRoot.sh` **on both pods** sets up the central inventory.
- **Then** `root.sh` , on **racnode1-0 first and only then on racnode2-0**. On node 1 it creates the cluster, formats the OCR and voting files into `DATA` , and starts ASM. Node 2 then joins that cluster. Running both at once corrupts the setup.
- `root.sh` **writes outside** `/u01` **:** `/etc/oracle` , `/etc/oratab` , the `oracle-ohasd` systemd service. That matters again in Phase 14.

racnode1-0, then racnode2-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash` ; for racnode2-0: `kubectl -n rac exec -it racnode2-0 -- bash`

```
/u01/app/oraInventory/orainstRoot.sh
```

racnode1-0, then racnode2-0

(10-20 minutes on racnode1-0; start racnode2-0 only when it has finished)

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash` ; for racnode2-0: `kubectl -n rac exec -it racnode2-0 -- bash`

```
setsid bash -c '/u01/app/19c/grid/root.sh; echo "exit $?"' > /u01/poc-logs/rootsh.log 2>&1 < /dev/null &
sleep 5; tail -f $(ls -t /u01/app/19c/grid/install/root_*.log | head -1)
```

Wait for `CLSRSC-325: Configure Oracle Grid Infrastructure for a Cluster ... succeeded` , then press Ctrl+C. **Before** you start racnode2-0, run the setuid check below in racnode1-0. Run `root.sh` in racnode2-0 only after both the `CLSRSC-325` line and that check are good.

Check the setuid bits in racnode1-0 as soon as its `root.sh` has finished, before racnode2-0's `root.sh` ; then again in racnode2-0 after its own `root.sh` . This is the check that catches the `ORA-12547` / `DBT-05801` problem from the first build:

racnode1-0 after its root.sh, then racnode2-0 after its root.sh

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash` ; for racnode2-0: `kubectl -n rac exec -it racnode2-0 -- bash`

```
GH=/u01/app/19c/grid
ls -l $GH/bin/oracle $GH/bin/oradism
echo "setuid/setgid files: $(find $GH/bin -perm /6000 -type f | wc -l)"
```

Check: `bin/oracle` is `-rwsr-s--x grid oinstall` and about 440 MB; `bin/oradism` is `-rwsr-x--- root oinstall` ; the setuid/setgid count is **5** (seen on racnode1-0 in this build; anything above 0 with those two lines is fine). If `bin/oracle` has no `s` , stop and paste the output.

11.6 Configuration assistants (racnode1-0, 5–10 minutes)

What: creates the listeners and finishes the ASM configuration, then runs a final verification (CVU).

Expected result: in this lab only the CVU post-check failed (`INS-43080`), on container-environment findings (`PRVF-4009` , `PRVG-2031` / `2033` , `PRVH-1526`). `netca` and `asmca` completed. A CVU-only failure is a warning, not a broken cluster.

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
setsid bash -c 'su - grid -c "/u01/app/19c/grid/gridSetup.sh -executeConfigTools -responseFile /u01/poc-logs/gi.rsp -silent"; echo "exit $?"' > /u01/poc-logs/configtools.log 2>&1 < /dev/null &
tail -f /u01/poc-logs/configtools.log
```

When it prints an `exit` line, press Ctrl+C and blank the passwords:

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
sed -i -E 's/^(oracle.install.asm.(SYSASM|monitor)Password)=.*/\1=/' /u01/poc-logs/gi.rsp && echo "passwords blanked"
```

11.7 Check the cluster (racnode1-0)

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
su - grid -c "crsctl check cluster -all; olsnodes -n -s -t; crsctl stat res -t; crsctl stat res -init -t | grep -A1 haip; oifcfg getif; srvctl config scan; export ORACLE_SID=+ASM1; asmcmd lsdg; \$ORACLE_HOME/OPatch/opatch lspatches"
```

Check:

- CRS, CSS and EVM are online on **both** nodes, and `olsnodes` shows both as `Active`;
- `ora.cluster_interconnect.haip` is ONLINE;
- `oifcfg` shows `eth0 10.244.0.0 public`, `eth1 10.0.11.0 cluster_interconnect,asm` and `eth2 10.0.12.0 cluster_interconnect,asm`;
- the SCAN has two VIPs (the pod IPs);
- `DATA` is mounted, `EXTERN`, with a 4 MB AU and about 100 GB;
- `opatch lspatches` lists the 19.32 Release Update patches. That's the proof the gold image carried them.

Last step: quick-check the permissions, before the database install. This is the check that would have caught the first build's problem early:

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
G=/u01/app/19c/grid; stat -c '%n %U:%G %a' $G/bin/oracle $G/bin/oradism $G/bin/extjob $G/perl/bin/perl
```

You should see:

- `bin/oracle` : grid:oinstall 6751;
- `bin/oradism` : root:oinstall 4750;
- `bin/extjob` : root:oinstall 4750;

- `perl` : owned by root.

If `extjob` isn't `root:oinstall 4750` , set it: `chown root $G/bin/extjob && chmod 4750 $G/bin/extjob` . The verification in the first build expected exactly that.

PHASE 12

The RAC database

Result of this phase: the RAC database `ORCLCDB` is open on both pods (instances `ORCLCDB1` and `ORCLCDB2`). It has one pluggable database, `ORCLPDB` , and a service, `ORCLPDBSVC` , offered on both instances. The 19.32 patches are applied in every container, and memory is sized for the pod.

12.1 Unzip the database gold image (racnode1-0)

The same rules as the Grid home: `unzip -K` , and `SWAP_SPACE` commented out. The database gold image was made the same way as the Grid one, so expect the same: `bin/oracle` is a 0-byte placeholder and there are no setuid bits yet. The installer links `bin/oracle` in 12.2 and `root.sh` sets the setuid bits in 12.3, where they are checked.

`racnode1-0`

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
OH=/u01/app/oracle/product/19c/dbhome_1
su - oracle -c "unzip -K -q -o /stage/oracle-dbhome-19.32.zip -d $OH"
sed -i 's/^SWAP_SPACE=/#SWAP_SPACE=/' $OH/oui/oraparam.ini; chown oracle:oinstall $OH/oui/oraparam.ini
du -sh $OH; ls -l $OH/bin/oracle; ls $OH/runInstaller
```

Check: `bin/oracle` is 0 bytes and `runInstaller` is listed. If `bin/oracle` is large instead, the zip was made differently from the Grid one; stop and paste the output.

12.2 Install the software on both pods (racnode1-0, 10–20 minutes)

What: `runInstaller` with `INSTALL_DB_SWONLY` . It only installs the software, registers the home on both nodes and copies it to racnode2-0. The database itself is created by DBCA in 12.4. These are the doc's parameters.

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
setsid bash -c 'su - oracle -c "/u01/app/oracle/product/19c/dbhome_1/runInstaller -waitforcompletion -
silent \
 oracle.install.option=INSTALL_DB_SWONLY ORACLE_HOSTNAME=racnode1-0 UNIX_GROUP_NAME=oinstall \
 oracle.install.db.CLUSTER_NODES=racnode1-0,racnode2-0 INVENTORY_LOCATION=/u01/app/oraInventory \
 SELECTED_LANGUAGES=en ORACLE_HOME=/u01/app/oracle/product/19c/dbhome_1 ORACLE_BASE=/u01/app/oracle
oracle.install.db.InstallEdition=EE \
 oracle.install.db.OSDBA_GROUP=dba oracle.install.db.OSBACKUPDBA_GROUP=backupdba
oracle.install.db.OSDGDBA_GROUP=dgdba \
 oracle.install.db.OSKMDBA_GROUP=kmdba oracle.install.db.OSRACDBA_GROUP=racdba \
 SECURITY_UPDATES_VIA_MYORACLESUPPORT=false DECLINE_SECURITY_UPDATES=true"; echo "exit $?"' > /u01/poc-
logs/runinstaller.log 2>&1 < /dev/null &
tail -f /u01/poc-logs/runinstaller.log
```

Wait for `Successfully Setup Software` and `exit 0` or `exit 6`.

12.3 Database home root.sh (both pods)

racnode1-0, then racnode2-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash` ; for racnode2-0: `kubectl -n rac exec -it racnode2-0 -- bash`

```
/u01/app/oracle/product/19c/dbhome_1/root.sh
su - oracle -c '$ORACLE_HOME/OPatch/opatch lspatches'
ls -l /u01/app/oracle/product/19c/dbhome_1/bin/oracle; echo "setuid/setgid files: $(find
/u01/app/oracle/product/19c/dbhome_1/bin -perm /6000 -type f | wc -l)"
```

Check: `opatch lspatches` lists the OCW, Database and OJVM 19.32 Release Updates. The DB home is patched without any OPatch run of your own. `bin/oracle` shows `-rwsr-s--x` and the setuid/setgid count is **greater than 0**; if not, stop before DBCA.

12.4 Create the database with DBCA (racnode1-0, 30–60 minutes)

What: the doc's DBCA command:

- a container database `ORCLCDB` with one PDB, `ORCLPDB`;
- RAC on both nodes;
- data files in `+DATA`, with Oracle-managed file names;
- AL32UTF8.

One change from the doc: `-totalMemory 3600` **instead of** `-memoryPercentage 30`. The doc's option needs patch **34436514**, which makes DBCA size memory from the pod's limit instead of the host's RAM. That fix is **not** in 19.32. On the first build, `-memoryPercentage 30` gave SGA+PGA of 6,498 MB, which is 30% of the 22 GB worker. The SGA (5.2 GB) didn't fit the pod's 4 GiB of hugepages, so the rest came out of the 12 GiB pod limit, where it competes with ASM and Grid. That's a risk of the pod being killed for running out of memory, which RAC sees as a node crash. 3,600 MB is 30% of the pod's 12 GiB.

The password must follow the same rules as the Grid one.

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
read -rsp "SYS / SYSTEM / PDB admin password: " PW; echo
export PW
setsid bash -c 'su - oracle -c "/u01/app/oracle/product/19c/dbhome_1/bin/dbca -silent -createDatabase -
templateName General_Purpose.dbc -gdbname ORCLCDB \
 -createAsContainerDatabase true -sysPassword $PW -systemPassword $PW -datafileDestination +DATA -
storageType ASM \
 -characterSet AL32UTF8 -redoLogFileSize 1024 -databaseType OLTP -databaseConfigType RAC \
 -nodelist racnode1-0,racnode2-0 -useOMF true -numberOfPDBs 1 -pdbAdminPassword $PW -pdbName ORCLPDB \
 -totalMemory 3600"; echo "exit $?"' > /u01/poc-logs/dbca.log 2>&1 < /dev/null &
unset PW
tail -f /u01/poc-logs/dbca.log
```

The progress stays on one percentage for long stretches. Real activity shows in the alert log, from a second window:

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl -n rac exec -it racnode1-0 -- tail -f
/u01/app/oracle/diag/rdbms/orclcdb/ORCLCDB1/trace/alert_ORCLCDB1.log
```

What you'll see in the alert log, and why it's fine: DBCA builds from a seed template that has a 19.3 data dictionary. The first time `PDB$SEED` opens, it reports **SQL Patch violations**: the 19.32 RU and OJVM are "installed in the CDB but not in the PDB". Near the end, DBCA runs **datapatch**, which applies the SQL part of the patches to every container, and the violations clear. That's the one patching step a gold image still needs: the binaries come pre-patched, but each new database's dictionary is patched once.

Wait for `100% complete` and `exit 0`.

12.5 The application service (racnode1-0)

Why a service: applications should connect to a **service**, not to an instance or to the CDB. `ORCLPDBSVC` runs in `ORCLPDB` and is preferred on both instances, so the SCAN listeners balance it across the nodes and move it if one fails. This is the doc's step.

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
su - oracle -c "srvctl add service -d ORCLCDB -pdb ORCLPDB -service ORCLPDBSVC -notification true -
preferred ORCLCDB1,ORCLCDB2; srvctl start service -d ORCLCDB -s ORCLPDBSVC; srvctl status database -d
ORCLCDB -verbose"
```

12.6 Check the database (racnode1-0)

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
su - oracle -c "export ORACLE_SID=ORCLCDB1; sqlplus -s / as sysdba" <<'SQL'
set lines 200 pages 100
col name for a24
col value for a30
col description for a55
select inst_id, instance_name, host_name, status from gv$instance order by 1;
select inst_id, name, open_mode from gv$pdbs order by 1, 2;
select name, value from v$parameter where name in ('sga_target','pga_aggregate_target','use_large_pages')
order by 1;
select con_id, patch_id, status, description from cdb_registry_sqlpatch order by con_id, action_time;
select name, cause, status from pdb_plug_in_violations where status <> 'RESOLVED';
SQL
```

Check:

- both instances are `OPEN`, on racnode1-0 and racnode2-0;
- `ORCLPDB` is `READ WRITE` on both instances, and `PDB$SEED` is `READ ONLY`;
- `sga_target` plus `pga_aggregate_target` come to about 3.6 GB, and `use_large_pages` is `TRUE`;
- RU 39472050 and OJVM 39222882 show `SUCCESS` for con_id 1 (the root) and con_id 3 (`ORCLPDB`);
- **no rows** of unresolved plug-in violations.

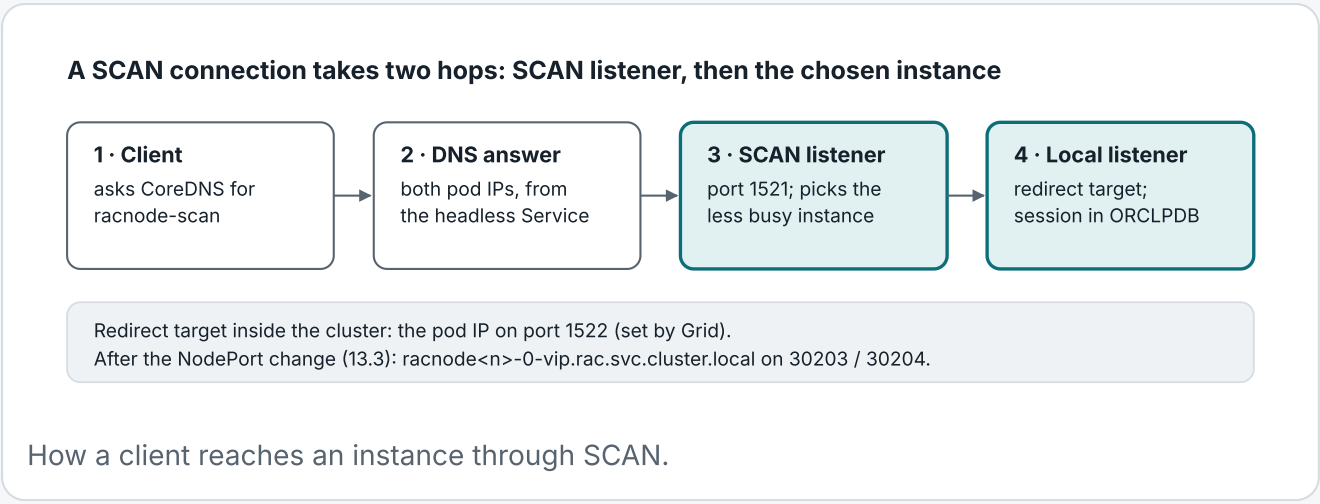
`cdb_` views leave out `PDB$SEED` by default, so the empty violations list is what proves the seed is patched too.

PHASE 13

Client connections inside and outside the cluster

Result of this phase: clients connect through the SCAN name and spread across both instances. That works from a pod in another namespace, and from Windows through NodePort services.

13.1 How a SCAN connection works here



How a client reaches an instance through SCAN.

1. **DNS lookup:** the client resolves `racnode-scan.rac.svc.cluster.local`. The headless Service returns **both pod IPs**.
2. **SCAN listener:** the client connects on port 1521 to one of the SCAN listeners (LISTENER_SCAN1 in racnode1-0, LISTENER_SCAN2 in racnode2-0).
3. **Redirect:** the SCAN listener knows which instances offer the service and how busy they are. It sends the client to that instance's **local listener**, at the address the instance registered in `local_listener`.
4. **Session:** the client reconnects there, and its session starts in `ORCLPDB`.

Step 3 is where Kubernetes setups break: the redirect address must be reachable by the client. Grid sets `local_listener` to the pod IP on port 1522, which any pod can reach.

13.2 Inside the cluster: a client pod in its own namespace

Why another namespace: an application will usually live in its own namespace. Using the full SCAN name proves the connection works across namespaces, which is how the doc describes it.

Why 1.5 GiB of memory: with 512 MiB, `dnf` was killed while loading the repository metadata (exit 137).

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl create namespace rac-client
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata: { name: dbclient, namespace: rac-client }
spec:
  nodeSelector: { kubernetes.io/hostname: worker1 }
  containers:
  - name: client
    image: container-registry.oracle.com/os/oraclelinux:9
    imagePullPolicy: IfNotPresent
    command: ["sleep", "infinity"]
    resources:
      requests: { cpu: 100m, memory: 256Mi }
      limits:   { cpu: 500m, memory: 1536Mi }
EOF
kubectl -n rac-client wait --for=condition=Ready pod/dbclient --timeout=120s
kubectl -n rac-client exec dbclient -- bash -c 'dnf -y -q install oracle-instantclient-release-23ai-el9
&& dnf -y -q install oracle-instantclient-sqlplus && sqlplus -V'
```

Now a small test that asks for the password once and opens six separate connections through SCAN. The password reaches SQL*Plus on stdin, never on a command line.

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
cat <<'XEOF2' | kubectl -n rac-client exec -i dbclient -- bash -c 'cat > /root/scan-test.sh'
read -rsp "SYSTEM password: " PW; echo
for i in 1 2 3 4 5 6; do
  sqlplus -s /nolog <<EOF | grep -v -e '^$' -e '^Connected'
set head off feed off
connect system/"$PW"@//racnode-scan.rac.svc.cluster.local:1521/ORCLPDBSVC
select 'connection $i: '||sys_context('USERENV','INSTANCE_NAME')||' on
'||sys_context('USERENV','SERVER_HOST')||' in '||sys_context('USERENV','CON_NAME') from dual;
exit
EOF
done
XEOF2
kubectl -n rac-client exec -it dbclient -- bash /root/scan-test.sh
```

Check: six lines such as `connection 1: ORCLCDB2 on racnode2-0 in ORCLPDB`, with **both** instances appearing. An idle database can favour one instance, so run it again if every line shows the same one.

13.3 Outside the cluster: the doc's NodePort procedure

The problem: a client outside Kubernetes can't resolve `racnode-scan.rac.svc.cluster.local` and can't reach pod IPs. A NodePort service opens a port on every node and forwards it to the pods. But the SCAN listener still

redirects the client to the *local* listener, so the local listener must also be reachable on a NodePort, **on the same port number** in the pod as on the node.

The doc's three steps:

1. **A second node listener,** `db-lsnr`**,** on 30203 and 30204.
2. **Each instance's** `local_listener` set to its VIP's full name and its own port: racnode1-0 → 30203, racnode2-0 → 30204.
3. **Three NodePort services:** 30200 → the SCAN listeners on 1521, 30203 → racnode1-0, 30204 → racnode2-0, plus 30201/30202 for ONS (FAN notifications).

`racnode1-0`

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
su - grid -c "srvctl add listener -listener db-lsnr -endpoints 30203,30204; srvctl start listener -
listener db-lsnr; srvctl status listener"
su - oracle -c "export ORACLE_SID=ORCLCDB1; sqlplus -s / as sysdba" <<'SQL'
ALTER SYSTEM SET local_listener='(ADDRESS=(PROTOCOL=TCP)(HOST=racnode1-0-vip.rac.svc.cluster.local)
(PORT=30203))' SCOPE=BOTH SID='ORCLCDB1';
ALTER SYSTEM SET local_listener='(ADDRESS=(PROTOCOL=TCP)(HOST=racnode2-0-vip.rac.svc.cluster.local)
(PORT=30204))' SCOPE=BOTH SID='ORCLCDB2';
alter system register;
SQL
```

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: Service
metadata: { name: racnode-scan-lsnr, namespace: rac }
spec:
  type: NodePort
  ports: [ { port: 1521, nodePort: 30200, targetPort: 1521 } ]
  selector: { cluster: racnode }
---
apiVersion: v1
kind: Service
metadata: { name: racnode1-np, namespace: rac, labels: { cluster: racnode, scan_svc_np: racnode1-np } }
spec:
  type: NodePort
  ports:
  - { name: ons-np-30201, nodePort: 30201, port: 6200, protocol: TCP, targetPort: 6200 }
  - { name: nodelsnr-np-30203, nodePort: 30203, port: 30203, protocol: TCP, targetPort: 30203 }
  selector: { hostname: racnode1-0 }
---
apiVersion: v1
kind: Service
metadata: { name: racnode2-np, namespace: rac, labels: { cluster: racnode, scan_svc_np: racnode2-np } }
spec:
  type: NodePort
  ports:
  - { name: ons-np-30202, nodePort: 30202, port: 6200, protocol: TCP, targetPort: 6200 }
  - { name: nodelsnr-np-30204, nodePort: 30204, port: 30204, protocol: TCP, targetPort: 30204 }
  selector: { hostname: racnode2-0 }
EOF
kubectl -n rac get svc | grep -E 'NAME|np|lsnr'
```

Check that inside the cluster still works: run `scan-test.sh` again. Clients are now redirected to the VIP names on ports 30203 and 30204 instead of to IPs on 1522, which they resolve through CoreDNS.

13.4 The lab's path from Windows

The lab sits behind VirtualBox NAT, so Windows needs two more things:

- **Port forwards** for 30200, 30203 and 30204 to worker1. Any node works, because kube-proxy sends a NodePort to the right pod wherever it runs.
- **Hosts-file entries** for the two VIP names, pointing at 127.0.0.1. The SCAN redirect names them, and Windows can't resolve cluster names.

PowerShell

```
$VB = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
foreach ($p in 30200, 30203, 30204) { & $VB natnetwork modify --netname k8s-public --port-forward-4
"np$($p):tcp:[]:$($p):[10.0.1.11]:$($p)" }
```

PowerShell (Admin)

```
Add-Content -Path "$env:windir\System32\drivers\etc\hosts" -Value "`r`n127.0.0.1 racnode1-0-
vip.rac.svc.cluster.local`r`n127.0.0.1 racnode2-0-vip.rac.svc.cluster.local"
ipconfig /flushdns
```

The full path:

1. Python connects to `127.0.0.1:30200`.
2. The port forward sends that to worker1's port 30200.
3. The NodePort passes it to a SCAN listener.
4. The SCAN listener redirects the client to `racnode1-0-vip.rac.svc.cluster.local:30203`.
5. The hosts file maps that name to 127.0.0.1.
6. The port forward and the NodePort carry it to `db-lsnr` in racnode1-0.

13.5 Connect from Windows

Python's `oracledb` driver in **thin mode** needs no Oracle client installed. Save this as `scan_test.py`:

python

```
import getpass, sys
import oracledb

dsn = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1:30200/ORCLPDBSVC"
pw = getpass.getpass("SYSTEM password: ")
sql = ("select sys_context('USERENV','INSTANCE_NAME'), sys_context('USERENV','SERVER_HOST'),"
       " sys_context('USERENV','CON_NAME') from dual")
for i in range(1, 7):
    try:
        with oracledb.connect(user="system", password=pw, dsn=dsn, tcp_connect_timeout=10) as conn:
            inst, host, con = conn.cursor().execute(sql).fetchone()
            print(f"connection {i}: {inst} on {host} in {con}")
    except oracledb.Error as e:
        print(f"connection {i}: FAILED {e}")
```

PowerShell

```
pip install oracledb
python .\scan_test.py
```

Check: six connections spread across ORCLCDB1 and ORCLCDB2.

If it fails:

- `WinError 10061` **("actively refused") on 127.0.0.1:** a port forward or the NodePort is missing.
- **A failure on 30203 or 30204, or a name error:** the redirect can't be followed. Check the hosts file.

PHASE 14

Surviving pod restarts

Result of this phase: a RAC pod that is deleted, crashes or loses its worker comes back by itself and rejoins the cluster, with the instance open. Clients carry on through the other instance in the meantime.

14.1 Why a restart breaks a RAC pod

When a pod is recreated (deleted, its container restarted, or its worker rebooted), Kubernetes builds a **new container from the image**. Only the volumes survive. `/u01` is a volume, so the Grid home, the database home and the inventory are safe. But Grid's root scripts also wrote files into the container's own filesystem, and those are gone:

- `/etc/oracle/` : `ocr.loc` and `olr.loc` (where the cluster registries are), `scls_scr/<node>/` (the node's start state), and `setasmgid` ;
- **the start-up machinery:** `/etc/init.d/init.ohasd` , `/etc/init.d/ohasd` , and the `oracle-ohasd` systemd service with its enable links;
- `/etc/oratab` , `/etc/oraInst.loc` , and `/usr/local/bin/oraenv` , `coraenv` and `dbhome` ;
- **the grid and oracle home directories** (SSH keys, `known_hosts` , `.bashrc`), and the pod's SSH host keys.

What it looks like: the new pod is `Running` but stays `0/1` , and `crsctl check crs` says `CRS-4047: No Oracle Clusterware components configured` .

What happens in the meantime: node 1 evicts node 2, recovers its unfinished transactions, and carries on. Clients reconnect through SCAN to the surviving instance. No data is lost: the database is on the shared disks.

The pod usually gets a new IP (10.244.2.3, then .4, then .5 in the tests). Grid copes because it stores the VIPs as **names** (`racnode2-0-vip` , `racnode2-0`) and resolves them again through DNS each time a resource starts.

14.2 The doc's manual procedure

This is the doc's "Complete Pod Restart Configuration Tasks", run as root in the recreated pod (racnode2-0 here).

`racnode2-0` (the recreated pod)

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode2-0 -- bash`

```
G=/u01/app/19c/grid
$G/crs/install/tfa_setup -silent -crshome $G      # doc step 1: diagnostics framework (not needed for Grid to start)
$G/crs/install/rootcrs.sh -updateosfiles          # doc step 2: Grid regenerates /etc/oracle, init scripts, systemd unit
$G/bin/crsctl start crs -wait                     # doc step 3
$G/bin/crsctl stat res -t                         # doc step 4
```

Then the doc's SSH and environment steps: the keys from `/mnt/.ssh` , `ssh-keyscan` on **both** pods, an SSH check, and the `.bashrc` lines. These are the same commands as 11.1.

What the first build found:

- `rootcrs.sh -updateosfiles` **doesn't recreate** `/etc/oratab` , `/etc/oraInst.loc` **or** `oraenv` , and the doc doesn't mention them. OPatch needs `oraInst.loc` , so patching that node would fail later. Copy them from the other pod:

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl -n rac exec racnode1-0 -- tar -C / -cpf - etc/oraInst.loc etc/oratab usr/local/bin/dbhome
usr/local/bin/oraenv usr/local/bin/coraenv | kubectl -n rac exec -i racnode2-0 -- tar -C / -xpf -
```

In this setup node 1's `oratab` has no entries: Grid and the database are managed by the cluster registry, not by `oratab`. If yours lists `+ASM1`, change it to `+ASM2` on node 2.

- `tfa_setup` **failed with** `AHF-00026: Perl is not owned by root`. That was a leftover of the recursive `chown`. It doesn't stop Grid.

14.3 Automating it

The doc's own note recommends backing up the home directories and other root-filesystem content under `/u01`, so they can be restored. This lab does exactly that, and runs the doc's steps automatically at every pod start:

1. `snapshot.sh` saves everything listed in 14.1 into `/u01/app/restart-state/os-files.tgz`, keeping owners, permissions and setuid bits. It refuses to overwrite a good snapshot with an incomplete one, and keeps the previous one as `os-files.tgz.prev`.
2. **The image's** `setupContainerEnv.sh` already calls `/u01/app/restart-state/onstart.sh` if it exists (Phase 8).
3. `onstart.sh` checks whether the `oracle-ohasd` service is missing, which means the pod was recreated. If so, in the background, it:
4. restores the home directories, SSH host keys, `oratab`, `oraInst.loc` and `oraenv`;
5. restarts `sshd`;
6. runs `tfa_setup`;
7. runs `rootcrs.sh -updateosfiles`, falling back to the snapshot's copy if that fails;
8. runs `crsctl start crs -wait`;
9. logs everything to `/u01/app/restart-state/onstart.log`.

racnode1-0, then racnode2-0

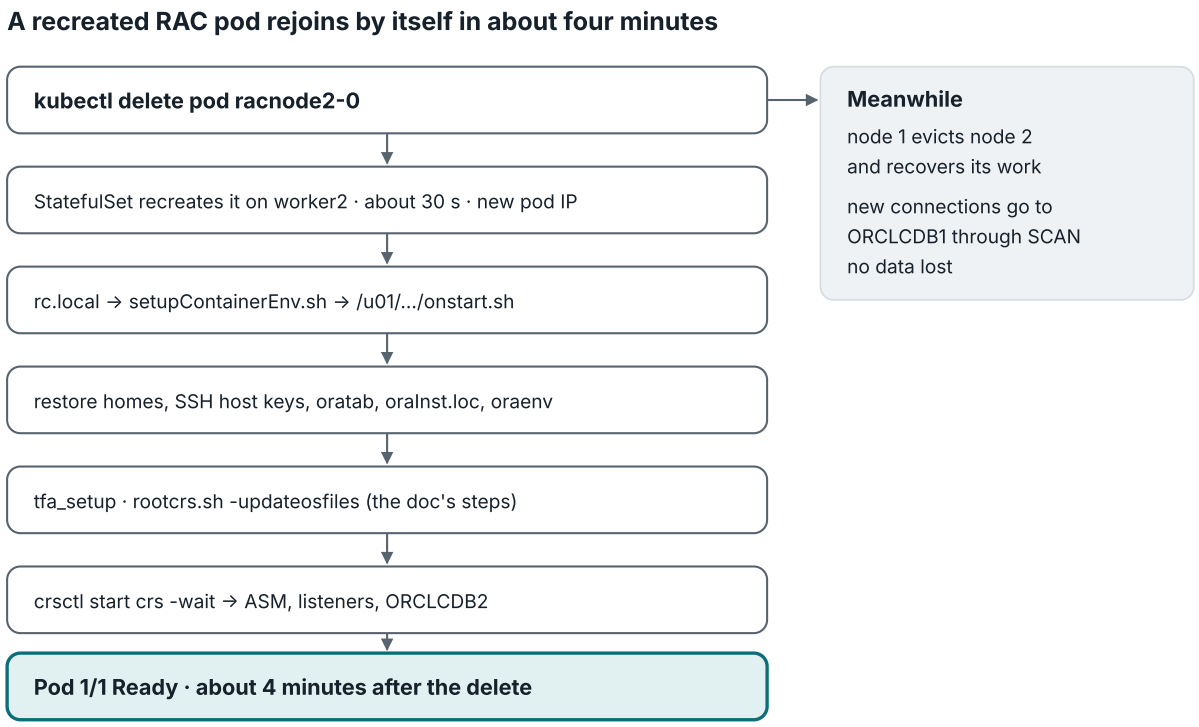
CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash` ; for racnode2-0: `kubectl -n rac exec -it racnode2-0 -- bash`

```
D=/u01/app/restart-state; mkdir -p $D
cat >$D/snapshot.sh <<'EOF'
#!/bin/bash
set -euo pipefail
D=/u01/app/restart-state; cd /
paths=(etc/oracle etc/oratab etc/oraInst.loc etc/init.d/init.ohasd etc/init.d/ohasd
       usr/local/bin/dbhome usr/local/bin/oraenv usr/local/bin/coraenv home/grid home/oracle)
paths+=( $(ls -d etc/ssh/ssh_host_* 2>/dev/null || true) )
paths+=( $(find etc/systemd -name 'oracle-ohasd*' 2>/dev/null) )
keep=(); for p in "${paths[@]}"; do [ -e "/$p" ] || [ -L "/$p" ] && keep+=("$p"); done
need=(etc/oracle/ocr.loc etc/oracle/olr.loc etc/oratab etc/oraInst.loc etc/init.d/init.ohasd
      etc/systemd/system/oracle-ohasd.service usr/local/bin/oraenv home/grid/.ssh/id_rsa
home/oracle/.ssh/id_rsa)
miss=(); for n in "${need[@]}"; do [ -e "/$n" ] || miss+=("/$n"); done
if [ ${#miss[@]} -gt 0 ] && [ "${1:-}" != "--force" ]; then echo "ERROR: missing ${miss[*]}; existing
snapshot kept"; exit 1; fi
tar --numeric-owner -czpf $D/os-files.tgz.new --exclude='etc/oracle/maps/*' "${keep[@]}"
if [ -f $D/os-files.tgz ]; then cp -p $D/os-files.tgz $D/os-files.tgz.prev; fi
mv -f $D/os-files.tgz.new $D/os-files.tgz
tar -tzvf $D/os-files.tgz | awk '{print $1, $2, $6}' > $D/os-files.list; date -Is > $D/snapshot.time
echo "snapshot: $(wc -l < $D/os-files.list) entries, $(cat $D/snapshot.time)"
EOF
cat >$D/onstart.sh <<'EOF'
#!/bin/bash
D=/u01/app/restart-state; G=/u01/app/19c/grid; log=$D/onstart.log
if [ "$1" != "--run" ]; then
  [ -f /etc/systemd/system/oracle-ohasd.service ] && { echo "=== $(date -Is) Grid files present, nothing
to do" >> $log; exit 0; }
  setsid nohup "$0" --run >> $log 2>&1 < /dev/null &
  exit 0
fi
echo "=== $(date -Is) $(hostname): pod was recreated, running the restart tasks"
[ -s $D/os-files.tgz ] || { echo "no snapshot; do the doc's steps by hand"; exit 1; }
tar --numeric-owner -xzpf $D/os-files.tgz -C / --exclude='etc/oracle' --exclude='etc/oracle/*' --
exclude='etc/init.d/*ohasd' --exclude='etc/systemd/*'
systemctl restart sshd
$G/crs/install/tfa_setup -silent -crshome $G || echo "WARN tfa_setup failed"
if ! $G/crs/install/rootcrs.sh -updateosfiles || [ ! -f /etc/systemd/system/oracle-ohasd.service ]; then
  echo "WARN -updateosfiles failed: using the snapshot's copy"
  tar --numeric-owner -xzpf $D/os-files.tgz -C / etc/oracle etc/init.d $(tar -tzf $D/os-files.tgz | grep
'^etc/systemd/.*oracle-ohasd')
  systemctl daemon-reload; systemctl enable oracle-ohasd.service
fi
$G/bin/crsctl start crs -wait
$G/bin/crsctl stat res -t
echo "=== $(date -Is) restart tasks finished"
EOF
```

```
chmod 750 $D/snapshot.sh $D/onstart.sh
bash $D/snapshot.sh
```

Check: `snapshot: ... entries` in each pod. Rerun `snapshot.sh` after any change to the Grid or database configuration.

14.4 Test it



Automatic restart path (Phase 14.3). The doc's manual path runs the same steps by hand.

Run `scan_test.py` from Windows, and leave it ready to run again.

```
controlplane
CONNECT FIRST  ssh -p 2210 root@127.0.0.1

kubectl -n rac delete pod racnode2-0 --wait=false
kubectl -n rac get pods -o wide -w
```

In another window, as soon as the new pod is `Running`, follow the restart log:

```
controlplane
CONNECT FIRST  ssh -p 2210 root@127.0.0.1

kubectl -n rac exec -it racnode2-0 -- tail -f /u01/app/restart-state/onstart.log
```

Check:

- **Right after the delete,** `scan_test.py` puts all six connections on ORCLCDB1, with no errors.

- **About 30 seconds later,** racnode2-0 is back with a new IP, showing `0/1`.
- **The log** shows the restore, `tfa_setup`, `CLSRSC-330 ... oracle-ohasd.service`, the `CRS-` start lines and the `crsctl stat res -t` table.
- **After about 4 minutes,** the pod is `1/1` with nothing typed. `scan_test.py` spreads across both instances again, and SSH between the pods works with no `ssh-keyscan`, because the original host keys were restored.

PHASE 15

Verify the whole lab

Result of this phase: proof, from top to bottom, that the lab matches the first build's final state.

15.1 Kubernetes

`controlplane`

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl get nodes -o wide
kubectl get pods -A -o wide | grep -v Running
kubectl -n multus-rac get network-attachment-definitions
kubectl get pv; kubectl -n rac get pvc,svc,pods -o wide
```

Check: 3 nodes `Ready`; no system pods that aren't `Running`; 2 NADs; 2 PVs and 2 PVCs `Bound`; the 5 headless services and 3 NodePort services; both RAC pods `1/1`.

15.2 Each worker

`worker1, then worker2`

CONNECT FIRST `ssh -p 2211 root@127.0.0.1` for worker1, `ssh -p 2212 root@127.0.0.1` for worker2

```
uname -r; cat /sys/kernel/mm/transparent_hugepage/enabled; getenforce; getsebool container_manage_cgroup
grep HugePages_Total /proc/meminfo; ip -br link show enp0s8; ip -br link show enp0s9
findmnt /scratch; findmnt /var/lib/containers
for d in $(ls /dev/disk/by-id/ | grep VBOX_HARDDISK | grep -v part); do echo "$d $(lsblk -dno SIZE
/dev/disk/by-id/$d) $(blkid -s TYPE -o value /dev/disk/by-id/$d)"; done
```

Check:

- UEK R7, THP `[madvise]`, `Enforcing`, and `container_manage_cgroup --> on`;
- 2048 hugepages, and both private NICs `UP` with `PROMISC`;
- both data disks mounted;
- the two 50 G disks show `oracleasm`. They're blank before the Grid install, and carry ASM's signature after it.

15.3 Grid and the Oracle homes

Why `cluvfy comp software`: it compares every file in a home with the owner, group and permissions recorded at install. It's the check that proved the homes clean after the first build's permissions repair. Before that repair, it reported 417 wrongly owned files on one node.

`racnode1-0`

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
su - grid -c "crsctl check cluster -all; crsctl stat res -t"
su - grid -c "cluvfy comp software -n all -verbose" | grep -E 'PRV|Software home'
su - oracle -c "/u01/app/19c/grid/bin/cluvfy comp software -d /u01/app/oracle/product/19c/dbhome_1 -n all
-verbose" | grep -E 'PRV|Software home'
```

Check:

- every resource is ONLINE on its home node;
- both homes report `Software home ... PASSED`, with no `PRV` lines.

15.4 Database, clients and restart

Rerun these checks:

- **12.6:** the instances, PDBs, memory, patches and violations;
- **13.2 and 13.5:** connections inside the cluster and from Windows;
- `bash /u01/app/restart-state/snapshot.sh` in each pod, so the snapshots match the final state.

That is the end state of the first build: all five PoC success criteria met.

Demo Q&A

These are questions a Kubernetes or Oracle audience is likely to ask, with answers based on what this lab showed.

Kubernetes questions

Why StatefulSets and not a Deployment? A RAC node needs a stable host name, because Grid registers it in the cluster registry. A StatefulSet gives `racnode1-0` every time; a Deployment would give a random name after every restart. There is one StatefulSet per node, so each can be pinned to its own worker.

Why is each pod pinned to one worker? Its `/u01` (the Oracle homes and the node's local registry) is a directory on that worker. The pod can't move without its software and identity. RAC provides the availability, not Kubernetes rescheduling.

Why headless Services? Grid needs DNS names that resolve to the **real pod addresses**, for the node, the VIP and the SCAN. A normal ClusterIP service would hand out a virtual IP that Grid can't bring up or check. `publishNotReadyAddresses` makes the names resolve before the pods are Ready, which is when the installer needs them.

Why Multus? RAC needs a private interconnect separate from the client network. A pod normally has one interface; Multus adds `eth1` and `eth2` on dedicated networks. macvlan gives each pod its own MAC address on the worker's NICs, and two networks give Grid's HAIP redundancy.

Why raw block volumes? ASM manages disks itself, with no filesystem. `volumeMode: Block` hands the pod the device. The same disks go to both pods (`ReadWriteMany`), and ASM coordinates access between the instances.

Isn't it insecure? The pods aren't privileged, but they get extra capabilities (`NET_ADMIN` , `SYS_NICE` , `SYS_RESOURCE` and others), namespaced "unsafe" sysctls, and hostPath volumes. In production you would put them on dedicated, tainted nodes in a namespace with explicit pod-security exceptions.

What happens when a pod dies? Clients carry on through the other instance straight away. Kubernetes recreates the pod on the same worker in about 30 seconds. Without extra handling, Grid wouldn't start, because files outside `/u01` are lost. This lab restores them and runs Oracle's restart steps automatically, and the node is back in about 4 minutes.

Oracle questions

Do the VIPs fail over? Not in the bare-metal sense. Each VIP is the pod's own IP, and Flannel routes a pod's address only to its worker, so it can't move. Clients survive a node loss through **SCAN**: the name resolves to the surviving node, and connect-time failover (plus FAN/ONS events) moves them.

The pod got a new IP after a restart. Doesn't that break Grid? No. Grid stores the VIP and SCAN VIP as **names** (`USR_ORA_VIP=racnode2-0-vip`) and resolves them through Kubernetes DNS each time it starts them. This was tested three times, with three different IPs.

Is it really patched to 19.32? Yes. `opatch lspatches` shows the RU in both homes. `cdb_registry_sqlpatch` shows the 19.32 RU and OJVM as `SUCCESS` in the root and the PDB. There are no unresolved PDB plug-in violations, which proves the seed is patched too.

What did the doc not cover?

- **DBCA memory:** `-memoryPercentage` sizes from the worker's RAM unless patch 34436514 is applied, and that fix isn't in 19.32.
- **The restart steps** don't recreate `/etc/oratab` , `/etc/oraInst.loc` or `oraenv` .
- `cluvfy` needs plain (unhashed) `known_hosts` entries.
- **The Multus networks** need `"ips": true` for fixed addresses.
- **On kubeadm,** you set the SELinux boolean `container_manage_cgroup` yourself.

Is this supported for production? Oracle's certification matrix lists RAC 19c (19.28 and later) on Kubernetes on Oracle Linux 8 and 9. The only install guide it links is the OCNE one, and it doesn't say whether other Kubernetes distributions are covered. On kubeadm, get written confirmation from Oracle before running production. The build guide's "Production considerations" section lists what else would change.

Lab questions

Why VirtualBox and not Hyper-V? RAC needs disks shared by two VMs and promiscuous NICs for macvlan. VirtualBox has both: shareable fixed-size disks and Allow All mode. Hyper-V on Windows 10/11 can't share a virtual disk between VMs.

Why does the PC need a hosts file and port forwards? The lab is behind VirtualBox NAT. Port forwards reach the NodePorts, and the hosts file lets Windows resolve the VIP names that the SCAN listener redirects clients to.

Built from the lab's build log and scripts (rac-k8s-poc). Values are from the author's lab; change them to match yours.